

Report for the Degree of Master of Computer Science

Library Management System

Karan Tiwari (LC00015003132)

MCS 2nd Semester

Advanced Java Programming

Lincoln University College

Computer Science & Multimedia Department

Lincoln University, Malaysia

31st August, 2026

COVER LETTER

To,

The Head of Department

Lincoln International College

Dhobidhara, Kathmandu

Subject: Submission of Project Report on “Library Management System (LMS)”

Dear Sir/Madam,

I am pleased to submit my project report titled “Library Management System” as part of the partial fulfilment of the requirements for the Master’s in Computer Science course Advanced Java Programming. This report presents the analysis, design, development and implementation of a Spring Boot REST API that automates the management of a library’s catalogue, its members and its borrowing operations.

Through this project I have gained practical experience in backend development using Java 17, Spring Boot 3, Spring Data JPA, Spring Security with JSON Web Tokens, Spring AOP and Spring Scheduling, together with an H2 relational database for persistent storage. Beyond the mandatory CRUD requirements, the system implements five additional features: file upload and download, a scheduled overdue-detection job with e-mail notification, PDF report generation, CSV report generation and a persistent audit trail. I sincerely hope that this report meets your expectations and demonstrates the knowledge and skills I have acquired throughout the course.

I would like to express my heartfelt gratitude to my course instructor for Advanced Java Programming, Mr. Shyam Sundar Khatiwada, for his continuous guidance, encouragement and insightful feedback during this project. I also extend my appreciation to all faculty members and peers who supported me throughout the process.

Thank you for your time and consideration.

Yours sincerely,

Karan Tiwari (LC00015003132)

Date : 31/08/2026

ACKNOWLEDGEMENT

I would like to express my deepest gratitude to my project supervisor Mr. Shyam Sundar Khatiwada and Mr. Pratik Maharjan for their continuous guidance, encouragement and valuable feedback throughout the development of this project. Their expertise and insights were instrumental in shaping my understanding of layered backend architecture, secure API design and system integration.

I would also like to thank my teachers for providing me with the knowledge and technical foundation necessary to complete this work successfully. My sincere appreciation goes to my classmates for their constant motivation, collaboration and constructive discussions that helped improve this project.

Finally, I extend heartfelt thanks to my family for their unwavering support, patience and inspiration during the entire project development process. Their encouragement helped me stay dedicated and focused on achieving my objectives.

This project stands as a reflection of the combined efforts, mentorship and support of everyone who contributed directly or indirectly to its successful completion.

ABSTRACT

The Library Management System is a Spring Boot based RESTful web application developed to automate and simplify the management of a library's catalogue, its registered members and its book circulation. Traditional library management processes often depend on manual record keeping or basic spreadsheet software, which results in inefficient data management, difficulty in searching records, duplication of information, uncontrolled access to sensitive operations and no reliable record of who changed what. The proposed system addresses these problems by providing a structured, secure and maintainable backend application for managing library resources.

The system is developed using Java 17 and Spring Boot 3.2 and follows a strict four-layer architecture consisting of Controller, Service, Repository and Entity components. It provides complete CRUD operations for books, authors, categories, members and loans. For data persistence the application uses a file-based H2 relational database together with Spring Data JPA, which maps Java entity classes to relational tables and removes the need to write repetitive SQL manually. Server-side input validation is implemented with Jakarta Bean Validation, and a centralised exception handler translates application errors into meaningful, consistently structured HTTP responses.

Security is implemented using Spring Security with stateless JSON Web Token authentication. Passwords are hashed using BCrypt, and three roles — ADMIN, LIBRARIAN and MEMBER — control which operations each user may perform. The catalogue can be searched, filtered, sorted and paginated through Spring Data Specifications and Pageable parameters.

In addition to the mandatory requirements, the application implements five advanced features: book cover image upload and download, a daily scheduled task that detects overdue loans and dispatches reminder e-mails, PDF report generation using iText, CSV report generation using OpenCSV, and a persistent audit trail that records every write operation, reinforced by a Spring AOP aspect. The complete REST API is documented interactively through SpringDoc OpenAPI (Swagger UI).

Keywords: Spring Boot, Spring Data JPA, Spring Security, JSON Web Token, REST API, H2 Database, Spring AOP, Layered Architecture

TABLE OF CONTENTS

COVER LETTER	2
ACKNOWLEDGEMENT	3
ABSTRACT	4
TABLE OF CONTENTS	5
LIST OF FIGURES	8
LIST OF TABLES	10
ABBREVIATIONS	11
1. INTRODUCTION	12
2. LITERATURE REVIEW	13
3. SYSTEM ANALYSIS	14
3.1. Problem Definition	14
3.2. Proposed Solution	14
3.3. Use Case Diagram	16
3.4. Data Flow Diagram	17
3.5. Functional Requirements	18
3.6. Non-Functional Requirements	18
4. SYSTEM DESIGN	19
4.1. Architecture Diagram	19
4.2. Database Schema (ER Diagram)	20
4.3. Database Schema Summary	21
4.4. Technology Stack Justification	22
4.4.1. Backend: Java 17 & Spring Boot 3.2.5	22
4.4.2. Database: H2 (file-based)	22
4.4.3. Data Access: Spring Data JPA & Hibernate	22
4.4.4. Security: Spring Security & JSON Web Tokens	22

4.4.5. Validation & Error Handling: Jakarta Bean Validation	22
4.4.6. Reporting & Utilities: iText, OpenCSV, Spring Mail, Spring AOP	23
4.4.7. Documentation: SpringDoc OpenAPI (Swagger UI)	23
4.5. REST API Endpoint Design	24
5. IMPLEMENTATION	26
5.1. Description of Tools	26
5.1.1. Spring Boot	26
5.1.2. Spring Data JPA	26
5.1.3. Spring Security and JSON Web Tokens	26
5.1.4. H2 Database	26
5.1.5. SpringDoc OpenAPI and Swagger UI	26
5.1.6. Apache Maven	26
5.2. Project Structure	28
5.3. Application Configuration	28
5.4. Screenshots of Working Modules	30
5.4.1. Authentication and JWT Issuance	30
5.4.2. Role-Based Access Control	34
5.4.3. Input Validation	36
5.4.4. Global Exception Handling	38
5.4.5. CRUD Operations	40
5.4.6. Search, Filter, Sort and Pagination	47
5.4.7. Loan Issue, Return and Fine Calculation	51
5.4.8. File Upload and Download	54
5.4.9. Scheduled Task and E-mail Notification	55
5.4.10. PDF and CSV Report Generation	57
5.4.11. Audit Logging	59
5.4.12. API Documentation and Database Console	62
6. TESTING AND DEBUGGING	64

6.1. Test Cases	66
7. DISCUSSION	71
8. FUTURE ENHANCEMENTS AND CONCLUSION	73
8.1. Future Enhancements	73
8.2. Conclusion	73
9. REFERENCES	75

LIST OF FIGURES

Figure 3.1: Use Case Diagram	16
Figure 3.2: Level-1 Data Flow Diagram	17
Figure 4.1: System Architecture Diagram	19
Figure 4.2: Entity Relationship Diagram	20
Figure 5.1: Application configuration (application.properties)	29
Figure 5.2: Login screen of the web client	30
Figure 5.3: SecurityConfig.java — stateless filter chain and URL-level authorisation rules	31
Figure 5.4: JwtUtil.java — token generation, claim extraction and validation	31
Figure 5.5: JwtAuthFilter.java — validates the bearer token on every request	32
Figure 5.6: Successful login returning a signed JWT (200 OK)	32
Figure 5.7: Login with an incorrect password (401 Unauthorized)	33
Figure 5.8: Administrator dashboard — full navigation	34
Figure 5.9: Librarian dashboard — no access to audit logs	34
Figure 5.10: Member dashboard — catalogue browsing only	35
Figure 5.11: A MEMBER attempting to delete a book is refused (403 Forbidden)	35
Figure 5.12: A protected endpoint requested without a token (403 Forbidden)	35
Figure 5.13: Book.java — JPA mappings and Bean Validation constraints	36
Figure 5.14: Validation failure returning a field-by-field breakdown (400 Bad Request)	37
Figure 5.15: GlobalExceptionHandler.java — centralised error handling	38
Figure 5.16: Requesting a non-existent book (404 Not Found)	39
Figure 5.17: BookController.java — REST endpoints for the book resource	40
Figure 5.18: BookServiceImpl.java — business logic and entity/DTO mapping	41
Figure 5.19: Book catalogue in the web client	42
Figure 5.20: Creating a new book record	42
Figure 5.21: Creating a book (POST /api/books)	43
Figure 5.22: Updating a book (PUT /api/books/{id})	44

Figure 5.23: Deleting a book (DELETE /api/books/{id})	44
Figure 5.24: Retrieving a single book (GET /api/books/{id})	45
Figure 5.25: Author management	45
Figure 5.26: Category management	46
Figure 5.27: Member management	46
Figure 5.28: BookSpecification.java — composable, type-safe search predicates	47
Figure 5.29: Searching the catalogue by title	47
Figure 5.30: Filtering by category and availability	47
Figure 5.31: Paginated and sorted response with page metadata	48
Figure 5.32: Search by title keyword	49
Figure 5.33: Combined category filter, availability filter and descending sort	50
Figure 5.34: LoanServiceImpl.java — issue and return with automatic fine calculation	51
Figure 5.35: Loan register showing issued, returned and overdue loans	52
Figure 5.36: Issuing a book to a member	52
Figure 5.37: A loan returned six days late, with a fine of NPR 60 calculated automatically	53
Figure 5.38: Uploading a book cover image as multipart form-data	54
Figure 5.39: OverdueLoanScheduler.java — daily cron job with e-mail notification	55
Figure 5.40: LoanRepository.java — the JPQL query that identifies overdue loans	56
Figure 5.41: Application log showing three overdue loans detected and three reminders dispatched	56
Figure 5.42: ReportServiceImpl.java — PDF generation with iText	57
Figure 5.43: Report download page in the web client	57
Figure 5.44: Inventory report generated as CSV by OpenCSV	58
Figure 5.45: AuditAspect.java — Spring AOP advice intercepting service-layer write operations	59
Figure 5.46: Audit trail in the web client	60
Figure 5.47: Audit trail retrieved through the API	61
Figure 5.48: Swagger UI — API overview and authentication guide	62
Figure 5.49: Swagger UI — documented endpoints	63
Figure 5.50: H2 console — book catalogue joined with author and category	63

Figure 5.51: H2 console — persisted audit_log rows	64
--	----

LIST OF TABLES

Table 4.1: Database Schema Summary	21
Table 4.2: Principal REST API Endpoints and Access Control	25
Table 5.1: Source Code Package Structure	28
Table 6.1: Authentication and Authorisation Test Cases	66
Table 6.2: Book CRUD Test Cases	67
Table 6.3: Search, Filter, Sort and Pagination Test Cases	68
Table 6.4: Loan and Circulation Test Cases	69
Table 6.5: Additional Feature Test Cases	70

ABBREVIATIONS

Abbreviation	Meaning
AOP	Aspect-Oriented Programming
API	Application Programming Interface
BCrypt	Blowfish-based password hashing function
CRUD	Create, Read, Update and Delete
CSV	Comma-Separated Values
DTO	Data Transfer Object
ER	Entity Relationship
HTTP	Hypertext Transfer Protocol
JDBC	Java Database Connectivity
JPA	Java Persistence API
JSON	JavaScript Object Notation
JWT	JSON Web Token
LMS	Library Management System
ORM	Object-Relational Mapping
PDF	Portable Document Format
RBAC	Role-Based Access Control
REST	Representational State Transfer
SQL	Structured Query Language
UI	User Interface

1. INTRODUCTION

Libraries are important institutions for storing, organising and providing access to knowledge and educational resources. As the number of books and registered members grows, maintaining library records manually becomes increasingly difficult. Traditional paper-based or spreadsheet-based approaches require considerable time for recording issues and returns, searching for a title, updating information and monitoring availability. They also tend to produce duplicate records, human errors and unauthorised modification of information, with no reliable way of establishing who made a change.

A computerised Library Management System reduces these problems by providing a single centralised platform for managing library resources. Such a system allows administrators and librarians to maintain accurate catalogue and membership information, and it allows members to search the catalogue quickly. The project presented in this report is a Library Management System implemented as a RESTful Spring Boot application, accompanied by a lightweight web client that consumes the same public API.

The application uses Spring Data JPA for all database operations. Spring Data JPA provides repository abstractions that remove most of the repetitive data-access code an application would otherwise require, and it supplies ready-made support for CRUD operations, derived query methods, dynamic criteria queries, pagination and sorting. A file-based H2 relational database is used as the persistence layer. H2 is well suited to this project because it requires no separate database server, yet it still allows the application to demonstrate genuine relational database concepts, foreign-key relationships and JPA-based persistence, and its contents survive application restarts.

Security is a central part of the system. Spring Security is used to authenticate users and to control access to protected resources. Spring Security separates authentication, which establishes the identity of a user, from authorisation, which determines what that identity is permitted to do. This project authenticates users with stateless JSON Web Tokens rather than server-side sessions: a client presents its credentials once, receives a signed token, and then supplies that token on every subsequent request. Three roles are implemented — ADMIN, LIBRARIAN and MEMBER — each with a distinct set of permissions.

The application also includes a number of capabilities beyond basic CRUD. The catalogue can be searched by title, filtered by author, category and availability, sorted on any field and returned one page at a time. Librarians can issue and return books, with overdue fines calculated automatically on return. Administrators and librarians can upload book cover images and export circulation and inventory reports as PDF and CSV files. A daily scheduled job detects overdue loans and notifies the members concerned, and every write operation is recorded in a persistent audit trail.

2. LITERATURE REVIEW

Library Management Systems have been studied and implemented using a wide variety of programming languages, architectures, databases and frameworks. The development of these systems has moved steadily from desktop and manual approaches towards web-based and service-oriented architectures.

Earlier library management applications commonly focused on basic operations such as maintaining book records, managing users and recording borrowing information. Many implementations used Java-based desktop applications together with relational databases such as MySQL. A recent 2026 study describing a Java and MySQL Library Management System focused on automating manual processes such as book management, student records, and issue and return operations, while also emphasising validation and database constraints. [1]

Web-based systems provide additional advantages because users can interact with the application over a network rather than being restricted to a single computer. A 2023 IEEE conference paper by Guo et al. presented a web-based Library Management System using Java, a browser/server architecture and MySQL. That work included functional requirement analysis, database ER design and systematic testing of the resulting system. [2]

Recent work on Spring Boot based library systems has demonstrated the use of REST APIs, layered architectures, relational persistence and security mechanisms. A 2025 study describes a Spring Boot based book management system built around a RESTful API, a controller-service-repository architecture, relational persistence and Spring Security. [3]

Another recent study specifically examined a Spring Boot Library Management System using object-oriented design and a four-layer controller-service-repository-entity architecture, highlighting the benefits of separation of concerns, maintainability and extensibility. [4]

A common limitation across much of this earlier work is the reliance on HTTP Basic authentication or server-side sessions, which do not scale well and couple the client tightly to the server. More recent literature on securing Spring Boot APIs recommends stateless token-based authentication using JSON Web Tokens, in which the server issues a signed token that the client presents on each request, allowing the server to remain free of session state. [5]

The present project builds on these findings. It implements a complete Spring Boot REST API with a clean four-layer architecture, but replaces Basic authentication with stateless JWT authentication and three-tier role-based authorisation. It also adds cross-cutting concerns that are frequently absent from student implementations — centralised exception handling, a scheduled background job, aspect-oriented audit logging and exportable reports — in order to demonstrate a broader range of Advanced Java capabilities within a small, comprehensible scope.

3. SYSTEM ANALYSIS

3.1. Problem Definition

Traditional library record management becomes difficult to sustain when catalogue, membership and circulation information is maintained manually or in unstructured spreadsheets. As the number of books and members grows, keeping records accurate, locating a particular title, updating information and controlling who may perform administrative operations all become progressively more time-consuming and error-prone.

The main problems identified for the proposed system are:

- **Manual and duplicated record keeping** — Book, author, category and member details are re-typed into several places, producing inconsistent and duplicated records with no single source of truth.
- **Difficulty in searching and retrieving records** — Locating a book by title, author, category or availability requires a manual scan of the whole register, which does not scale as the collection grows.
- **Lack of controlled access** — Shared spreadsheets and paper registers offer no way to restrict destructive operations, so any user who can read the records can also alter or delete them.
- **Weak or absent data validation** — Incomplete or invalid entries — a missing ISBN, a negative copy count, an impossible publication year — are accepted silently and corrupt the records.
- **Poor error handling** — Failures surface as raw technical errors that give the user no indication of what went wrong or how to correct it.
- **Manual circulation and overdue tracking** — Due dates must be checked by hand, overdue members are contacted inconsistently, and fines are calculated manually, which delays returns and loses revenue.
- **No audit trail** — When a record is altered or removed there is no reliable record of who performed the operation or when, which makes accountability impossible.
- **Inefficient handling of large result sets** — Returning an entire catalogue in a single response becomes slower as the collection grows, with no mechanism for paging through results.

3.2. Proposed Solution

The proposed solution is a Library Management System built with Spring Boot that provides a centralised and secure way to manage library records through REST APIs. The system uses a layered architecture consisting of controller, service, repository and entity layers, which makes the application easier to understand, test, maintain and extend. Each layer has a single responsibility:

controllers handle HTTP concerns, services hold business rules and transaction boundaries, repositories perform persistence, and entities model the domain.

Spring Data JPA is used to communicate with the relational H2 database, so that books, authors, categories, members, loans and users can be stored and related to one another efficiently. The system provides complete CRUD operations for every one of these entities. To make retrieval efficient, users can search books by title and filter them by author, category and availability, with the results sorted on any field and returned one page at a time using Spring Data Specifications combined with Pageable parameters.

Spring Security with stateless JSON Web Token authentication and role-based authorisation ensures that only appropriately privileged users can perform sensitive operations. Administrators may perform every operation including deletion and audit inspection; librarians may manage the catalogue, members and circulation; members may only browse and search the catalogue. Passwords are never stored in plain text — they are hashed with BCrypt before persistence.

Jakarta Bean Validation rejects invalid input before it reaches the database, and a centralised `@RestControllerAdvice` exception handler converts every failure into a consistent JSON error response carrying an appropriate HTTP status code and, for validation failures, a field-by-field breakdown of what was wrong. Circulation is automated through loan issue and return endpoints that adjust available copy counts and calculate overdue fines, supported by a daily scheduled job that flags overdue loans and notifies the members concerned. Finally, every create, update and delete operation is written to a persistent audit table so that all changes remain traceable.

3.3. Use Case Diagram

The use case diagram in Figure 3.1 identifies the actors that interact with the system and the operations available to each of them. Three human actors are defined — Admin, Librarian and Member — together with a system actor, the overdue scheduler, which acts on the system without human intervention. Permissions are cumulative: a Member may browse and search the catalogue, a Librarian may additionally manage the catalogue and circulation, and an Admin may perform every operation including deletion, user management and audit inspection.

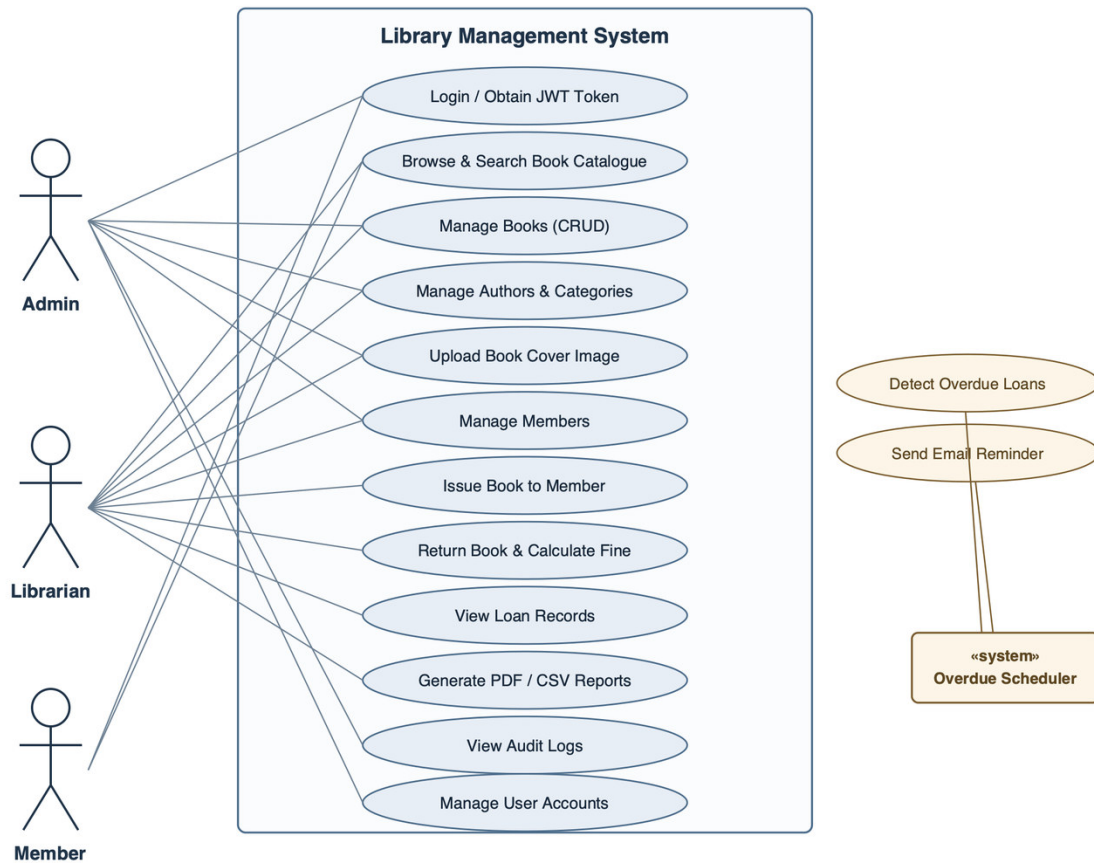


Figure 3.1: Use Case Diagram

3.4. Data Flow Diagram

Figure 3.2 shows the level-1 data flow diagram. The three external entities submit requests that are first processed by the authentication and authorisation process, which verifies the credentials against the users data store and returns a signed token. Subsequent requests flow to the catalogue, circulation or reporting processes, each of which reads from and writes to its own data stores. Every write operation additionally produces an entry in the audit logging process, which persists it to the audit_log store.

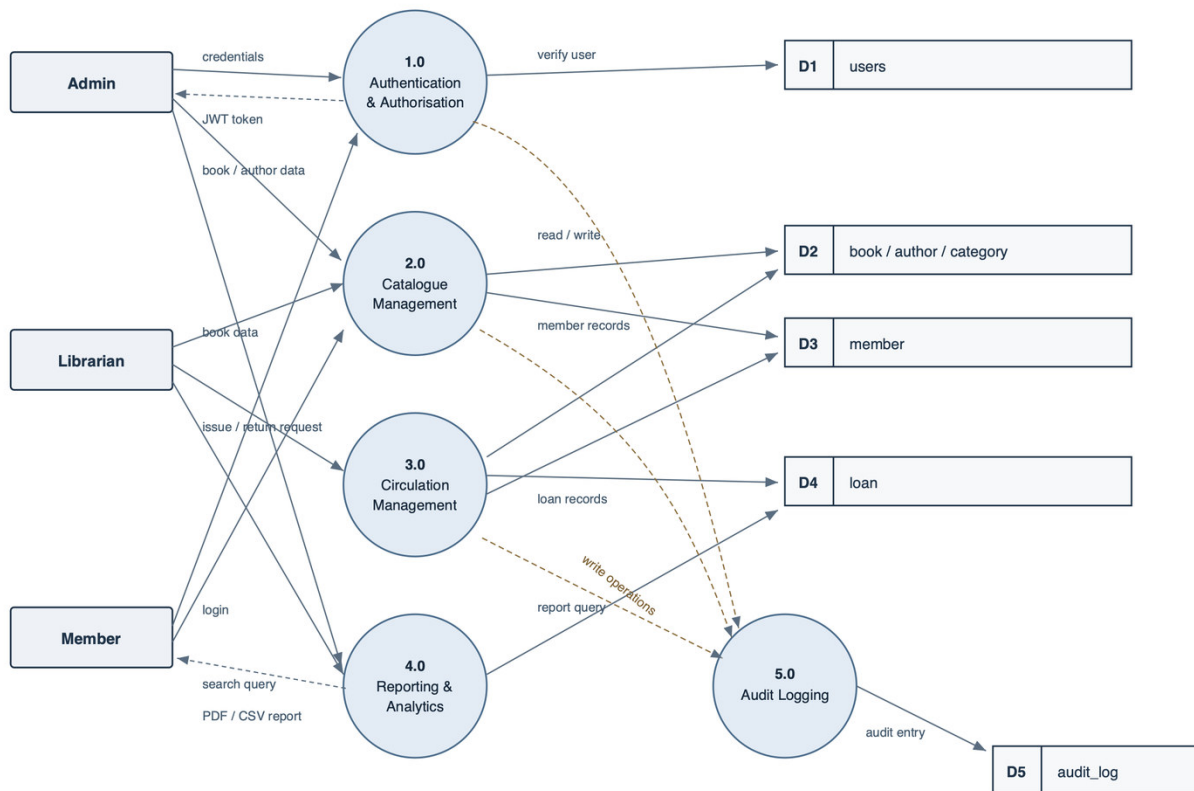


Figure 3.2: Level-1 Data Flow Diagram

3.5. Functional Requirements

- Secure login with username and password, returning a signed JWT valid for 24 hours.
- Role-based authorisation across three roles: ADMIN, LIBRARIAN and MEMBER.
- Complete Create, Read, Update and Delete operations for Book, Author, Category, Member and Loan.
- Search books by title and filter by author, category and availability.
- Sort and paginate all list endpoints.
- Server-side input validation with field-level error reporting.
- Centralised exception handling returning consistent, meaningful HTTP responses.
- Issue a book to a member, decrementing the available copy count.
- Return a book, restoring the copy count and calculating any overdue fine automatically.
- Upload and download a book cover image.
- Detect overdue loans daily and notify the affected members by e-mail.
- Export a circulation report as PDF and an inventory report as CSV.
- Record every write operation in a persistent audit trail.
- Publish interactive API documentation through Swagger UI.

3.6. Non-Functional Requirements

- **Security** — Passwords hashed with BCrypt; stateless JWT authentication; method- and URL-level role checks; CSRF disabled in favour of token authentication.
- **Performance** — Pagination on all list endpoints and lazy loading of associations keep response sizes and query cost bounded as the collection grows.
- **Maintainability** — Strict four-layer separation, DTOs at the API boundary, and interface-based services allow implementations to be replaced without affecting callers.
- **Portability** — H2 is accessed exclusively through JPA, so the application can be migrated to MySQL or PostgreSQL by changing configuration rather than code.
- **Usability** — A consistent ApiResponse envelope and interactive Swagger documentation make the API predictable and self-describing.
- **Traceability** — A persistent audit table plus an AOP logging aspect record who performed each write operation and when.

4. SYSTEM DESIGN

4.1. Architecture Diagram

The application follows a layered architecture in which each layer depends only on the layer immediately beneath it. A request arrives from a client — the bundled web interface, Swagger UI or an API tool — and passes first through the Spring Security filter chain, where the `JwtAuthFilter` validates the bearer token and establishes the security context. It then reaches a REST controller, which converts the payload into a DTO and delegates to a service. The service holds the business rules and the transaction boundary, and calls repositories to read and write entities. Spring Data JPA and Hibernate translate those calls into SQL against the H2 database. Cross-cutting concerns shown on the right of Figure 4.1 apply across all layers.

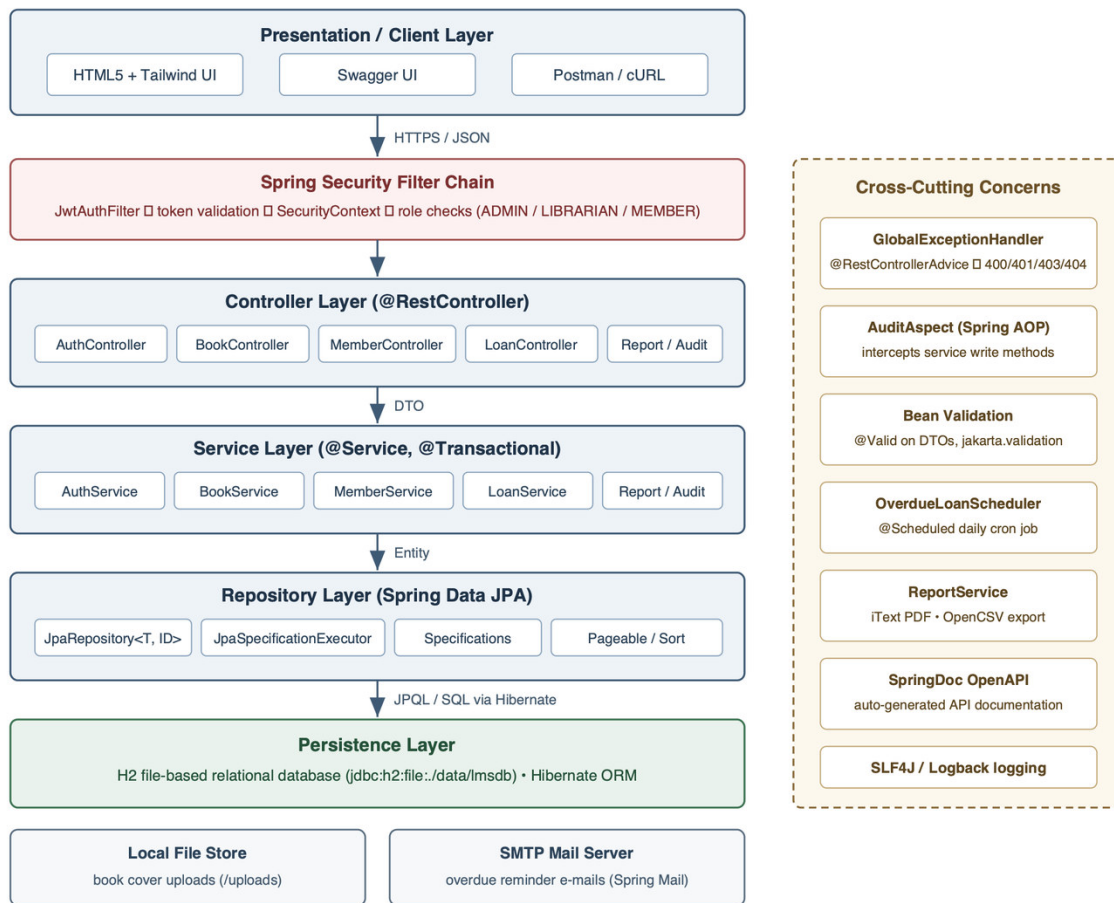


Figure 4.1: System Architecture Diagram

4.2. Database Schema (ER Diagram)

The schema is normalised to third normal form. Author and category are held as separate entities rather than repeated as text inside the book table, which removes the transitive dependencies that would otherwise cause update anomalies. The loan table acts as the associative entity linking a book to a member and carries the attributes that belong to the borrowing event itself — issue date, due date, return date, status and fine. The audit_log table records the acting username rather than a foreign key to users, so that the audit history survives even if a user account is later removed.

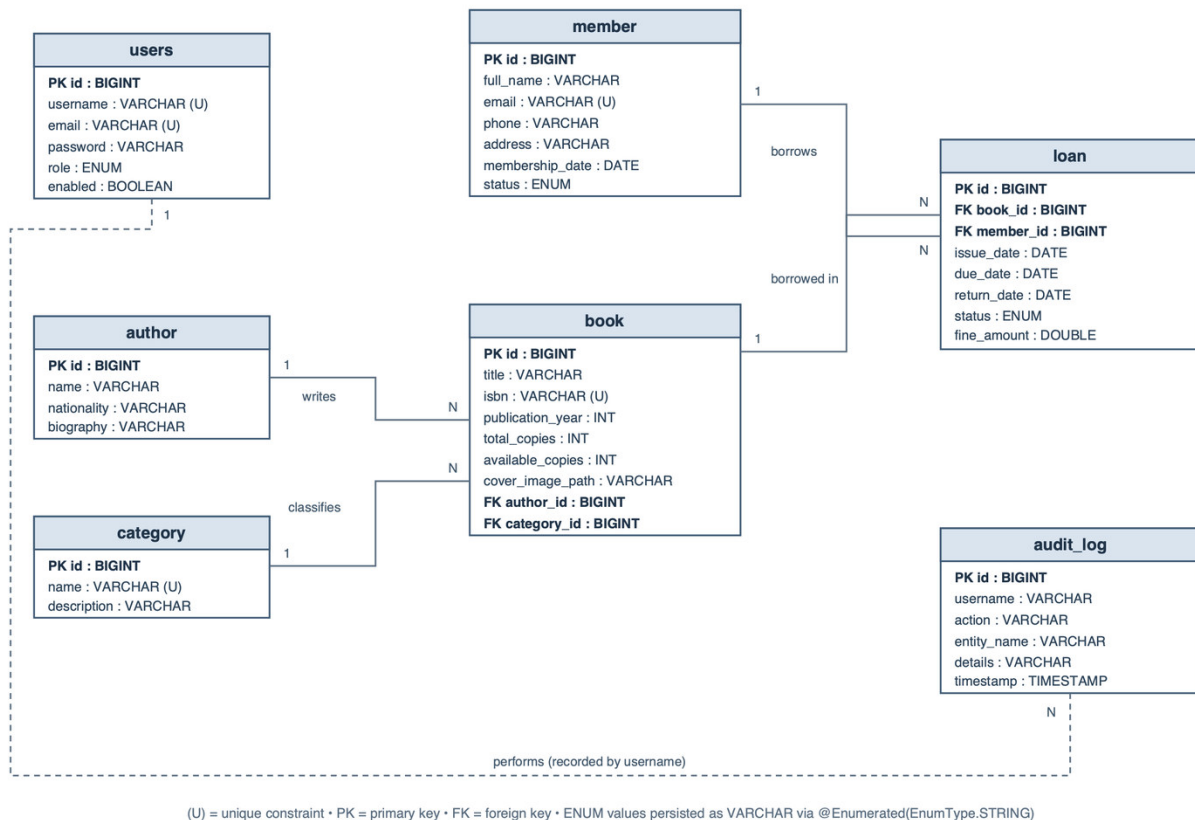


Figure 4.2: Entity Relationship Diagram

4.3. Database Schema Summary

Table 4.1: Database Schema Summary

Table	Columns	Purpose
users	id (PK), username (U), email (U), password, role, enabled, created_at	Application accounts. Passwords stored as BCrypt hashes; role is one of ADMIN, LIBRARIAN, MEMBER.
author	id (PK), name, nationality, biography	One author writes many books.
category	id (PK), name (U), description	One category classifies many books.
book	id (PK), title, isbn (U), publication_year, total_copies, available_copies, cover_image_path, description, author_id (FK), category_id (FK)	Catalogue entry. available_copies is decremented on issue and restored on return.
member	id (PK), full_name, email (U), phone, address, membership_date, status	Library patron. Status is ACTIVE, SUSPENDED or INACTIVE.
loan	id (PK), book_id (FK), member_id (FK), issue_date, due_date, return_date, status, fine_amount	Borrowing event. Status is ISSUED, RETURNED or OVERDUE; due date defaults to 14 days.
audit_log	id (PK), username, action, entity_name, details, timestamp	Immutable record of every create, update, delete and login operation.

4.4. Technology Stack Justification

4.4.1. Backend: Java 17 & Spring Boot 3.2.5

Spring Boot was chosen because it allows rapid development of production-ready REST APIs with minimal configuration. Its auto-configuration, embedded Tomcat server and starter dependencies remove most boilerplate, while its layered structure keeps the code reusable, easy to maintain and simple to debug. Java 17 is a long-term-support release and provides the language features and performance characteristics the framework expects. Because this system requires reliable CRUD operations, validation, transaction management and structured error handling, Spring Boot is a natural fit.

4.4.2. Database: H2 (file-based)

H2 was selected as the relational database because it is lightweight and requires no separate server installation, which suits an academic project that must be easy to build and demonstrate. Configured in file mode, it stores its data in `./data/lmsdb` so that records survive application restarts, and its browser console allows tables and rows to be inspected during development. Because all access goes through JPA, the system can be migrated to MySQL or PostgreSQL later by changing only the datasource configuration and dialect.

4.4.3. Data Access: Spring Data JPA & Hibernate

Spring Data JPA reduces the amount of SQL and data-access code that must be written by hand. It provides repository interfaces for common operations such as saving, finding, updating and deleting, and it derives queries from method names. Hibernate, acting as the JPA implementation, handles the conversion between Java objects and relational rows. For dynamic search this project additionally uses `JpaSpecificationExecutor`, which composes type-safe criteria predicates at runtime so that optional filters can be combined in any arrangement without writing a separate query for each combination.

4.4.4. Security: Spring Security & JSON Web Tokens

Spring Security provides the authentication and authorisation framework. This project uses stateless JWT authentication in preference to HTTP Basic authentication or server-side sessions: a custom `JwtAuthFilter` placed before the standard username/password filter validates the bearer token on each request and populates the security context. Because no session state is retained on the server, the API scales horizontally and can serve browser, mobile and tool-based clients identically. Passwords are hashed with BCrypt, which applies a per-password salt and a deliberately expensive work factor.

4.4.5. Validation & Error Handling: Jakarta Bean Validation

Validation constraints are declared directly on the DTO and entity fields and enforced by the framework before a controller method executes. A centralised `@RestControllerAdvice` converts

validation failures, missing resources, business-rule violations, authentication failures and authorisation failures into consistently structured JSON responses with the correct HTTP status code, so error handling is never duplicated across controllers.

4.4.6. Reporting & Utilities: iText, OpenCSV, Spring Mail, Spring AOP

iText generates the formatted PDF circulation report and OpenCSV streams the inventory report as CSV, both directly from the persistence layer. Spring Mail dispatches overdue reminders, and Spring's scheduling support runs the overdue detection job on a daily cron expression. Spring AOP supplies a logging aspect that intercepts every service-layer write method, which demonstrates a genuine cross-cutting concern implemented without polluting business logic.

4.4.7. Documentation: SpringDoc OpenAPI (Swagger UI)

SpringDoc inspects the controllers at runtime and publishes an OpenAPI 3.0 description of every endpoint together with an interactive Swagger UI. This gives the project living API documentation that cannot drift out of step with the code, and it provides a convenient way to exercise endpoints during demonstration and testing.

4.5. REST API Endpoint Design

The API follows REST conventions: resources are addressed by noun-based paths, the HTTP method expresses the operation, and every response is wrapped in a consistent `ApiResponse` envelope carrying a success flag, a human-readable message, the payload and a timestamp. Table 4.2 lists the principal endpoints together with the roles permitted to call them.

Table 4.2: Principal REST API Endpoints and Access Control

Method	Endpoint	Description	Access
POST	/api/auth/login	Authenticate and receive a JWT	Public
POST	/api/auth/register	Register a new user account	Public
GET	/api/books	Search, filter, sort and paginate books	Public
GET	/api/books/{id}	Retrieve a single book	Public
POST	/api/books	Create a book	ADMIN, LIBRARIAN
PUT	/api/books/{id}	Update a book	ADMIN, LIBRARIAN
DELETE	/api/books/{id}	Delete a book	ADMIN
POST	/api/books/{id}/cover	Upload a book cover image	ADMIN, LIBRARIAN
GET	/api/authors, /api/categories	List authors and categories	Public
GET	/api/members	Search and paginate members	ADMIN, LIBRARIAN
POST	/api/members	Register a library member	ADMIN, LIBRARIAN
POST	/api/loans/issue	Issue a book to a member	ADMIN, LIBRARIAN
PUT	/api/loans/{id}/return	Return a book and calculate any fine	ADMIN, LIBRARIAN
GET	/api/loans	List and paginate loans	ADMIN, LIBRARIAN
GET	/api/reports/loans/pdf	Download the circulation report as PDF	ADMIN, LIBRARIAN
GET	/api/reports/books/csv	Download the inventory report as CSV	ADMIN, LIBRARIAN
GET	/api/audit-logs	View the audit trail	ADMIN

5. IMPLEMENTATION

5.1. Description of Tools

5.1.1. Spring Boot

Spring Boot is a Java framework built on top of the Spring Framework that simplifies application development. It eliminates boilerplate through auto-configuration and opinionated starter dependencies, and it ships with an embedded server so that applications are production-ready out of the box. It supports web applications, REST APIs, microservices, security and cloud deployment. This project uses Spring Boot 3.2.5 on Java 17. [6]

5.1.2. Spring Data JPA

Spring Data JPA, part of the larger Spring Data family, makes it easy to implement JPA-based repositories. It significantly reduces the effort required to build a data access layer: the developer writes repository interfaces and Spring supplies the implementations automatically, deriving queries from method names or from explicit JPQL. In this project the repositories also extend JpaSpecificationExecutor so that dynamic, composable search criteria can be applied at runtime. [7]

5.1.3. Spring Security and JSON Web Tokens

Spring Security provides authentication, authorisation and protection mechanisms for Spring applications. This project configures a stateless filter chain in which a custom JwtAuthFilter extracts and validates the bearer token from the Authorization header before the request reaches a controller. Authorisation is enforced both at URL level, through requestMatchers in the security configuration, and implicitly by role, so that administrators, librarians and members each see a different set of permitted operations. [8]

5.1.4. H2 Database

H2 is the relational database used by the project. It is lightweight and requires no installation, which makes it well suited to an academic project and to demonstration. The application uses H2 in file mode so that data is retained between restarts, and the bundled H2 console makes it possible to inspect the generated schema and stored rows directly during development and testing.

5.1.5. SpringDoc OpenAPI and Swagger UI

SpringDoc OpenAPI generates an OpenAPI 3.0 specification from the running application and serves an interactive Swagger UI at /swagger-ui.html. Endpoints can be authorised with a JWT and invoked directly from the browser, which was used throughout development to test request bodies, authentication, validation errors and authorisation failures without a separate client.

5.1.6. Apache Maven

Maven is the build and dependency management tool for the project. The pom.xml declares the Spring Boot starters, the JWT, iText and OpenCSV libraries and the SpringDoc dependency, and the Spring Boot Maven plugin packages the application together with its embedded server into a single executable JAR.

5.2. Project Structure

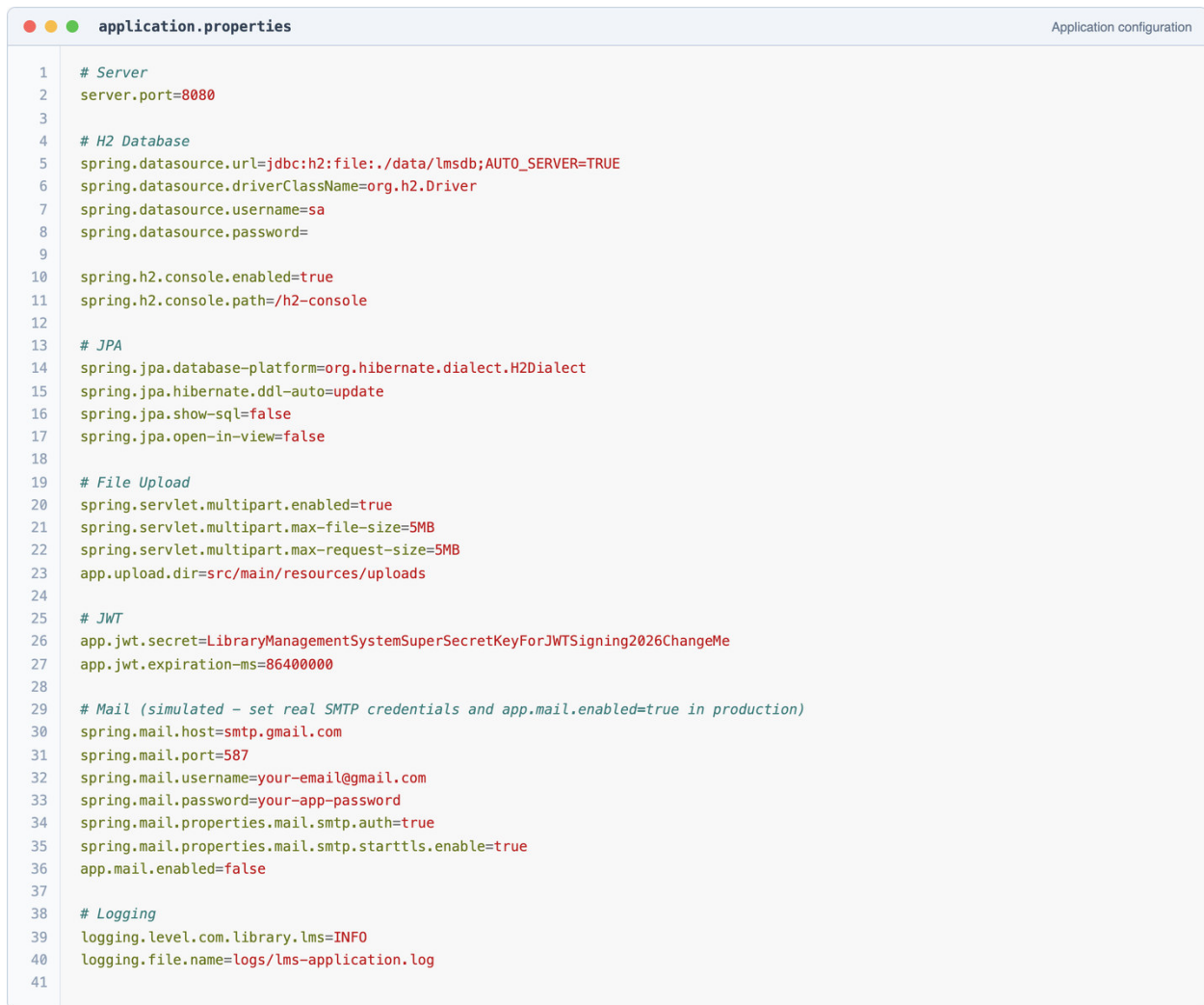
The source code is organised by responsibility rather than by feature, so that each package corresponds to one layer of the architecture described in Section 4.1.

Table 5.1: Source Code Package Structure

Package	Contents
entity/	JPA entities: Book, Author, Category, Member, Loan, User, Role, AuditLog
repository/	Spring Data JPA repositories, extending JpaSpecificationExecutor for dynamic search
specification/	BookSpecification and MemberSpecification — composable search predicates
dto/	Request and response DTOs carrying Bean Validation constraints
service/	Service interfaces defining the business contract
service/impl/	Service implementations holding business logic and transaction boundaries
controller/	REST controllers exposing the HTTP API
security/	JwtUtil, JwtAuthFilter and UserDetailsServiceImpl
config/	SecurityConfig, WebConfig, OpenApiConfig and DataSeeder
exception/	Custom exceptions and the GlobalExceptionHandler
scheduler/	OverdueLoanScheduler — the daily cron job
aspect/	AuditAspect — the Spring AOP logging aspect

5.3. Application Configuration

Figure 5.1 shows application.properties. Note in particular the file-based H2 URL, which causes the database to persist between restarts; spring.jpa.open-in-view set to false, which releases the database connection as soon as the service layer completes; and the JWT secret and 24-hour expiry used to sign and validate tokens.

The image shows a code editor window titled "application.properties" with a subtitle "Application configuration". The editor contains a list of configuration properties for a Spring application, organized into sections. The properties are as follows:

```
1 # Server
2 server.port=8080
3
4 # H2 Database
5 spring.datasource.url=jdbc:h2:file:./data/lmsdb;AUTO_SERVER=TRUE
6 spring.datasource.driverClassName=org.h2.Driver
7 spring.datasource.username=sa
8 spring.datasource.password=
9
10 spring.h2.console.enabled=true
11 spring.h2.console.path=/h2-console
12
13 # JPA
14 spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
15 spring.jpa.hibernate.ddl-auto=update
16 spring.jpa.show-sql=false
17 spring.jpa.open-in-view=false
18
19 # File Upload
20 spring.servlet.multipart.enabled=true
21 spring.servlet.multipart.max-file-size=5MB
22 spring.servlet.multipart.max-request-size=5MB
23 app.upload.dir=src/main/resources/uploads
24
25 # JWT
26 app.jwt.secret=LibraryManagementSystemSuperSecretKeyForJWTSigning2026ChangeMe
27 app.jwt.expiration-ms=86400000
28
29 # Mail (simulated - set real SMTP credentials and app.mail.enabled=true in production)
30 spring.mail.host=smtp.gmail.com
31 spring.mail.port=587
32 spring.mail.username=your-email@gmail.com
33 spring.mail.password=your-app-password
34 spring.mail.properties.mail.smtp.auth=true
35 spring.mail.properties.mail.smtp.starttls.enable=true
36 app.mail.enabled=false
37
38 # Logging
39 logging.level.com.library.lms=INFO
40 logging.file.name=logs/lms-application.log
41
```

Figure 5.1: Application configuration (application.properties)

5.4. Screenshots of Working Modules

This section documents each implemented module with the source code that provides it and the observed runtime behaviour. All screenshots were captured from the running application against seeded data.

5.4.1. Authentication and JWT Issuance

Authentication is stateless. The client posts its credentials to `/api/auth/login`; the service authenticates them against the BCrypt-hashed password held in the users table and, on success, returns a signed JSON Web Token together with the username and role. The client then sends that token in the Authorization header of every subsequent request.

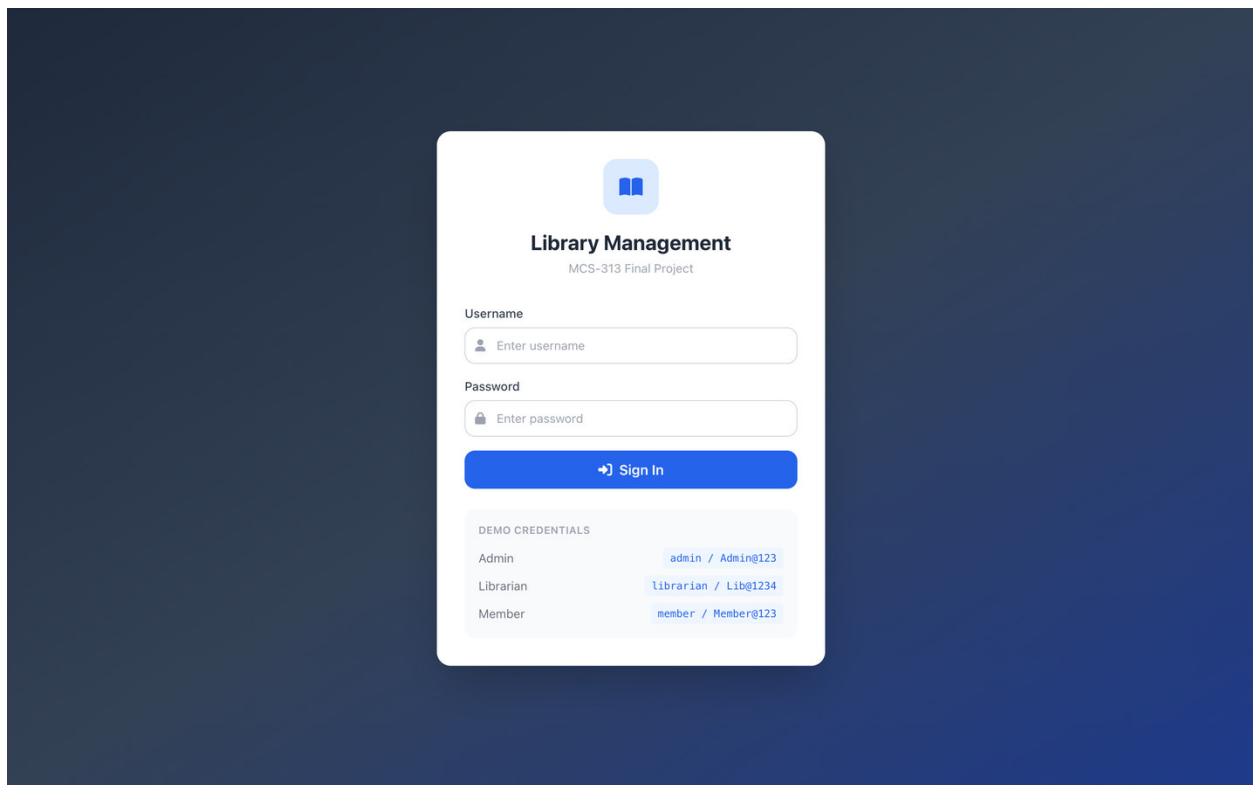


Figure 5.2: Login screen of the web client

```

SecurityConfig.java
Spring Security filter chain

56     .cors(cors -> cors.configurationSource(corsConfigurationSource()))
57     .csrf(csrf -> csrf.disable())
58     .sessionManagement(sm -> sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
59     .authorizeHttpRequests(auth -> auth
60         .requestMatchers("/", "/index.html", "/js/**", "/css/**", "/favicon.ico").permitAll()
61         .requestMatchers("/swagger-ui/**", "/swagger-ui.html", "/v3/api-docs/**").permitAll()
62         .requestMatchers("/api/auth/**", "/h2-console/**", "/uploads/**", "/api/public/**").permitAll()
63         .requestMatchers(HttpMethod.GET, "/api/books/**", "/api/authors/**", "/api/categories/**").permitAll()
64         .requestMatchers("/api/reports/**").hasAnyRole("ADMIN", "LIBRARIAN")
65         .requestMatchers("/api/audit-logs/**").hasRole("ADMIN")
66         .requestMatchers(HttpMethod.POST, "/api/books/**", "/api/authors/**", "/api/categories/**").hasAnyRole("ADMIN",
"LIBRARIAN")
67         .requestMatchers(HttpMethod.PUT, "/api/books/**", "/api/authors/**", "/api/categories/**").hasAnyRole("ADMIN",
"LIBRARIAN")
68         .requestMatchers(HttpMethod.DELETE, "/api/books/**", "/api/authors/**", "/api/categories/**").hasRole("ADMIN")
69         .requestMatchers("/api/members/**").hasAnyRole("ADMIN", "LIBRARIAN")
70         .requestMatchers("/api/loans/**").hasAnyRole("ADMIN", "LIBRARIAN")
71         .anyRequest().authenticated()
72     )
73     .headers(headers -> headers.frameOptions(frame -> frame.disable())) // allow H2 console frames
74     .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);
75
76     return http.build();
77 }
78
79 @Bean
80 public CorsConfigurationSource corsConfigurationSource() {
81     CorsConfiguration configuration = new CorsConfiguration();
82     configuration.setAllowedOriginPatterns(List.of("*"));

```

Figure 5.3: SecurityConfig.java — stateless filter chain and URL-level authorisation rules

```

JwtUtil.java
JWT generation & validation

31     public String generateToken(UserDetails userDetails, String role) {
32         Map<String, Object> claims = new HashMap<>();
33         claims.put("role", role);
34         return Jwts.builder()
35             .setClaims(claims)
36             .setSubject(userDetails.getUsername())
37             .setIssuedAt(new Date())
38             .setExpiration(new Date(System.currentTimeMillis() + expirationMs))
39             .signWith(key(), SignatureAlgorithm.HS256)
40             .compact();
41     }
42
43     public String extractUsername(String token) {
44         return extractClaim(token, Claims::getSubject);
45     }
46
47     public <T> T extractClaim(String token, Function<Claims, T> resolver) {
48         Claims claims = Jwts.parserBuilder().setSigningKey(key()).build().parseClaimsJws(token).getBody();
49         return resolver.apply(claims);
50     }
51
52     public boolean isValidToken(String token, UserDetails userDetails) {
53         String username = extractUsername(token);
54         Date expiration = extractClaim(token, Claims::getExpiration);
55         return username.equals(userDetails.getUsername()) && expiration.after(new Date());
56     }
57 }

```

Figure 5.4: JwtUtil.java — token generation, claim extraction and validation

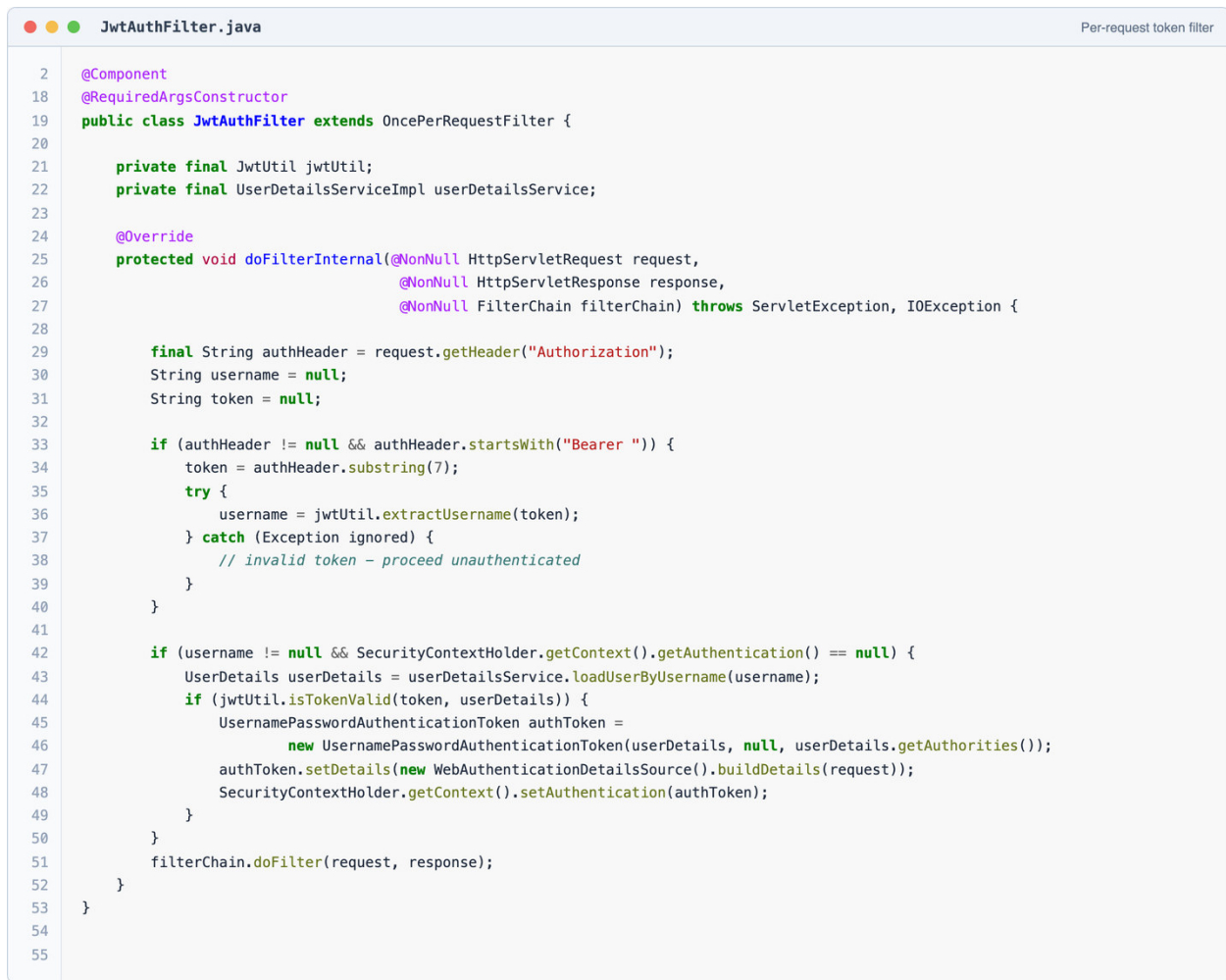


Figure 5.5: JwtAuthFilter.java — validates the bearer token on every request

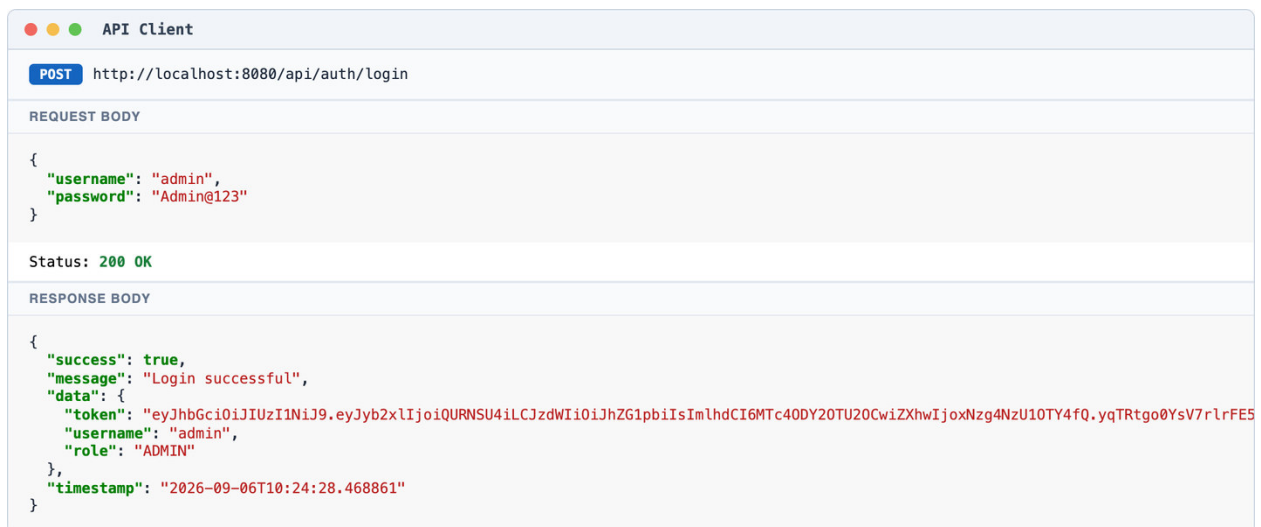


Figure 5.6: Successful login returning a signed JWT (200 OK)

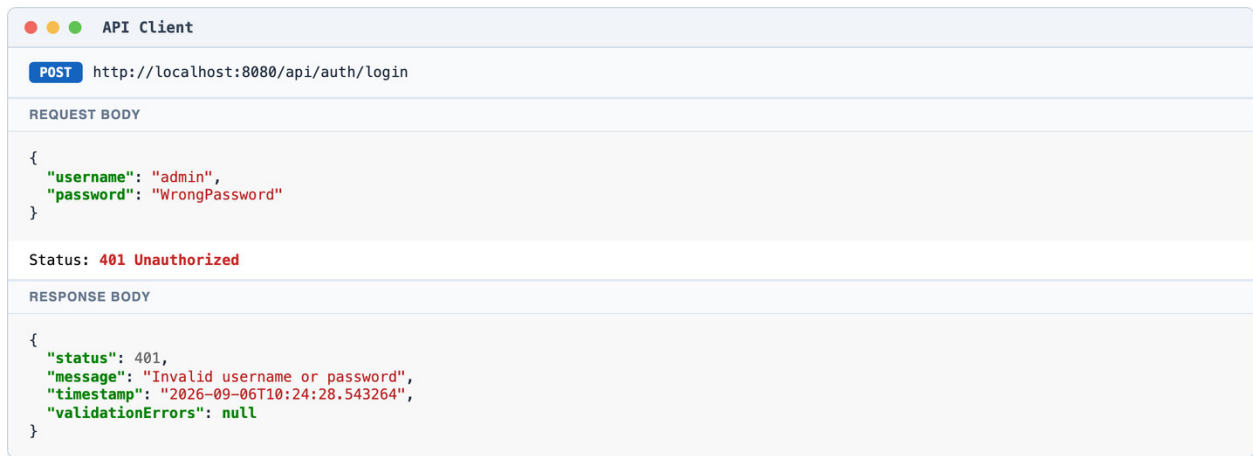


Figure 5.7: Login with an incorrect password (401 Unauthorized)

5.4.2. Role-Based Access Control

Three roles are implemented. An ADMIN may perform every operation; a LIBRARIAN may manage the catalogue, members and circulation but may not delete records or read the audit trail; a MEMBER may only browse and search the catalogue. The role is carried as a claim inside the JWT and enforced by the security configuration. The dashboards below show the same application rendering different navigation and different statistics for each role, and the API responses that follow show the server rejecting operations that exceed a caller's privileges.

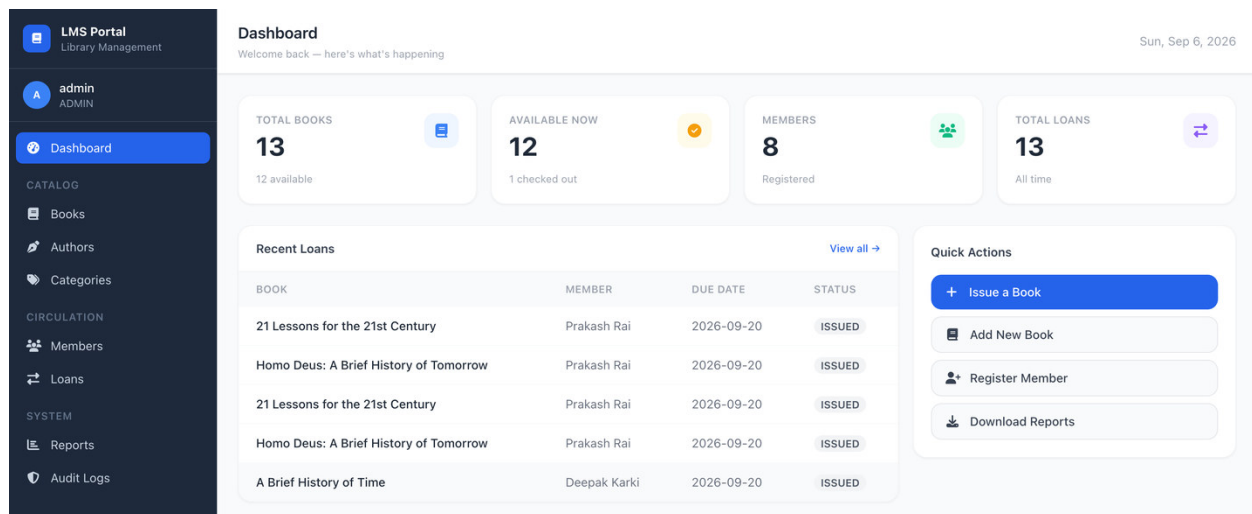


Figure 5.8: Administrator dashboard — full navigation

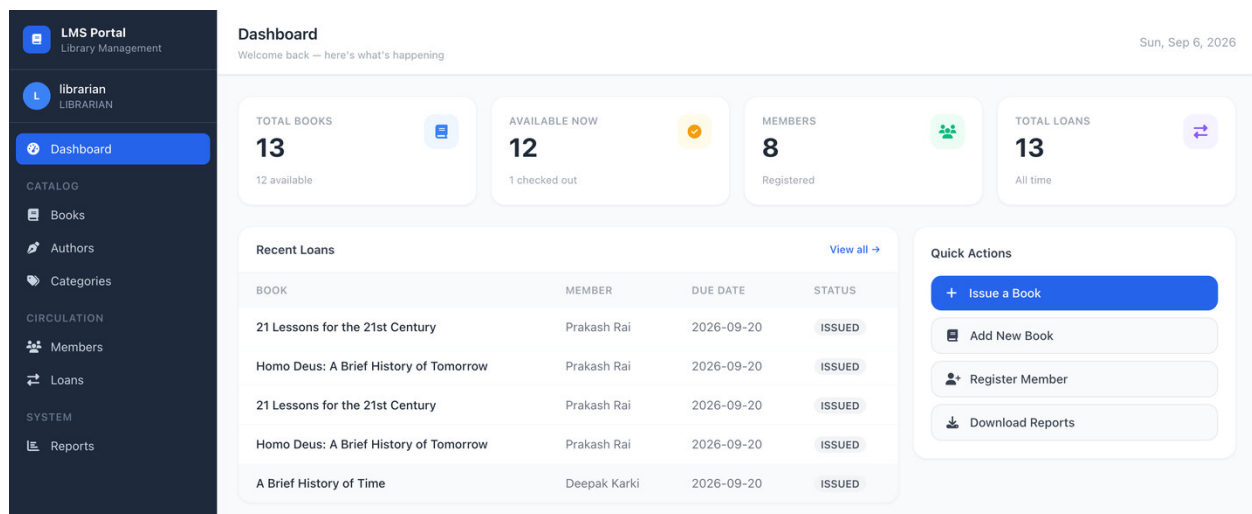


Figure 5.9: Librarian dashboard — no access to audit logs

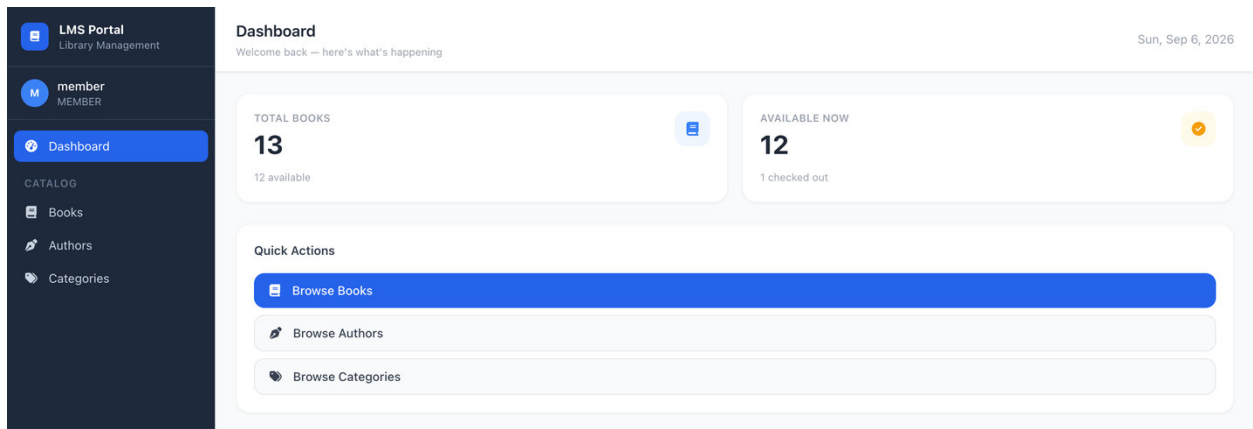


Figure 5.10: Member dashboard — catalogue browsing only

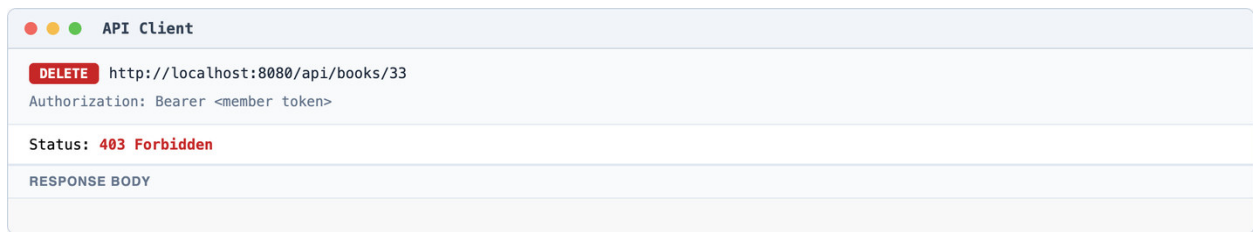


Figure 5.11: A MEMBER attempting to delete a book is refused (403 Forbidden)

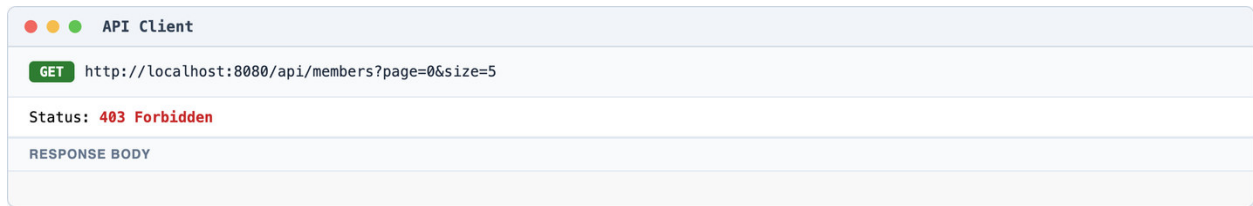
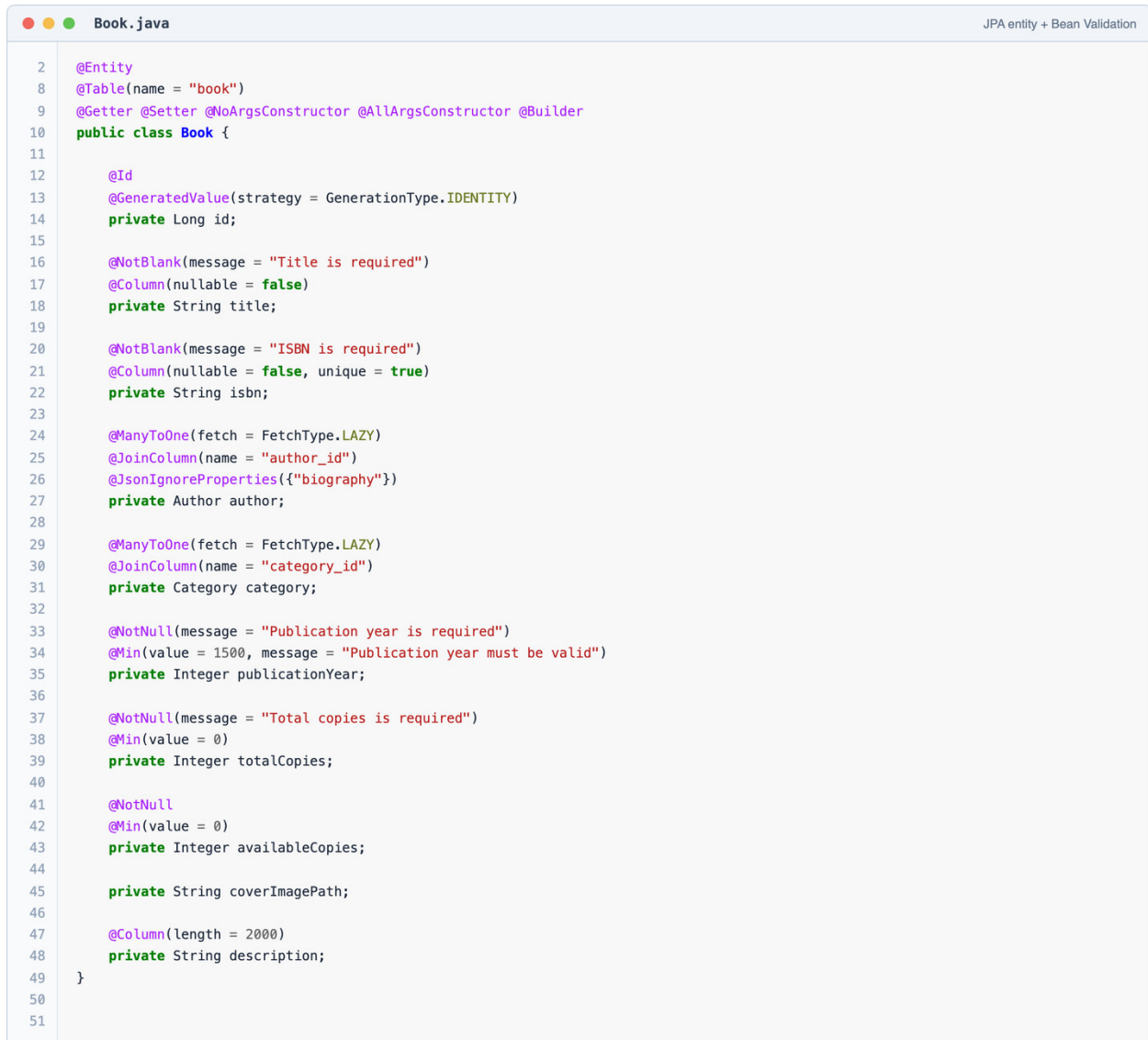


Figure 5.12: A protected endpoint requested without a token (403 Forbidden)

5.4.3. Input Validation

Validation constraints are declared directly on the entity and DTO fields using Jakarta Bean Validation annotations such as `@NotBlank`, `@NotNull`, `@Min`, `@Email` and `@Pattern`. Controllers mark their request bodies `@Valid`, so the framework enforces the constraints before the method body executes. When validation fails the request never reaches the service layer.



```
Book.java JPA entity + Bean Validation

2  @Entity
8  @Table(name = "book")
9  @Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
10 public class Book {
11
12     @Id
13     @GeneratedValue(strategy = GenerationType.IDENTITY)
14     private Long id;
15
16     @NotBlank(message = "Title is required")
17     @Column(nullable = false)
18     private String title;
19
20     @NotBlank(message = "ISBN is required")
21     @Column(nullable = false, unique = true)
22     private String isbn;
23
24     @ManyToOne(fetch = FetchType.LAZY)
25     @JoinColumn(name = "author_id")
26     @JsonIgnoreProperties({"biography"})
27     private Author author;
28
29     @ManyToOne(fetch = FetchType.LAZY)
30     @JoinColumn(name = "category_id")
31     private Category category;
32
33     @NotNull(message = "Publication year is required")
34     @Min(value = 1500, message = "Publication year must be valid")
35     private Integer publicationYear;
36
37     @NotNull(message = "Total copies is required")
38     @Min(value = 0)
39     private Integer totalCopies;
40
41     @NotNull
42     @Min(value = 0)
43     private Integer availableCopies;
44
45     private String coverImagePath;
46
47     @Column(length = 2000)
48     private String description;
49 }
50
51
```

Figure 5.13: Book.java — JPA mappings and Bean Validation constraints



Figure 5.14: Validation failure returning a field-by-field breakdown (400 Bad Request)

5.4.4. Global Exception Handling

A single `@RestControllerAdvice` centralises error handling for every controller. It maps missing resources to 404, business-rule violations and validation failures to 400, bad credentials to 401, authorisation failures to 403 and anything unforeseen to 500, and it emits the same `ErrorResponse` structure in each case. Because this logic lives in one class, no controller contains error-handling code.



```
2  @RestControllerAdvice
16 public class GlobalExceptionHandler {
17
18     @ExceptionHandler(ResourceNotFoundException.class)
19     public ResponseEntity<ErrorResponse> handleNotFound(ResourceNotFoundException ex) {
20         return build(HttpStatus.NOT_FOUND, ex.getMessage(), null);
21     }
22
23     @ExceptionHandler(BadRequestException.class)
24     public ResponseEntity<ErrorResponse> handleBadRequest(BadRequestException ex) {
25         return build(HttpStatus.BAD_REQUEST, ex.getMessage(), null);
26     }
27
28     @ExceptionHandler(MethodArgumentNotValidException.class)
29     public ResponseEntity<ErrorResponse> handleValidation(MethodArgumentNotValidException ex) {
30         Map<String, String> errors = new HashMap<>();
31         ex.getBindingResult().getFieldErrors().forEach(err ->
32             errors.put(err.getField(), err.getDefaultMessage()));
33         return build(HttpStatus.BAD_REQUEST, "Validation failed", errors);
34     }
35
36     @ExceptionHandler(BadCredentialsException.class)
37     public ResponseEntity<ErrorResponse> handleBadCredentials(BadCredentialsException ex) {
38         return build(HttpStatus.UNAUTHORIZED, "Invalid username or password", null);
39     }
40
41     @ExceptionHandler(AccessDeniedException.class)
42     public ResponseEntity<ErrorResponse> handleAccessDenied(AccessDeniedException ex) {
43         return build(HttpStatus.FORBIDDEN, "You do not have permission to perform this action", null);
44     }
45
46     @ExceptionHandler(MaxUploadSizeExceededException.class)
47     public ResponseEntity<ErrorResponse> handleMaxUpload(MaxUploadSizeExceededException ex) {
48         return build(HttpStatus.BAD_REQUEST, "File too large. Maximum allowed size is 5MB", null);
49     }
50
51     @ExceptionHandler(Exception.class)
52     public ResponseEntity<ErrorResponse> handleGeneric(Exception ex) {
53         return build(HttpStatus.INTERNAL_SERVER_ERROR, "An unexpected error occurred: " + ex.getMessage(), null);
54     }
55
56     private ResponseEntity<ErrorResponse> build(HttpStatus status, String message, Map<String, String> errors) {
57         ErrorResponse response = ErrorResponse.builder()
58             .status(status.value())
59             .message(message)
60             .timestamp(LocalDateTime.now())
61             .validationErrors(errors)
62             .build();
63         return ResponseEntity.status(status).body(response);
64     }
65 }
66
67
```

Figure 5.15: GlobalExceptionHandler.java — centralised error handling

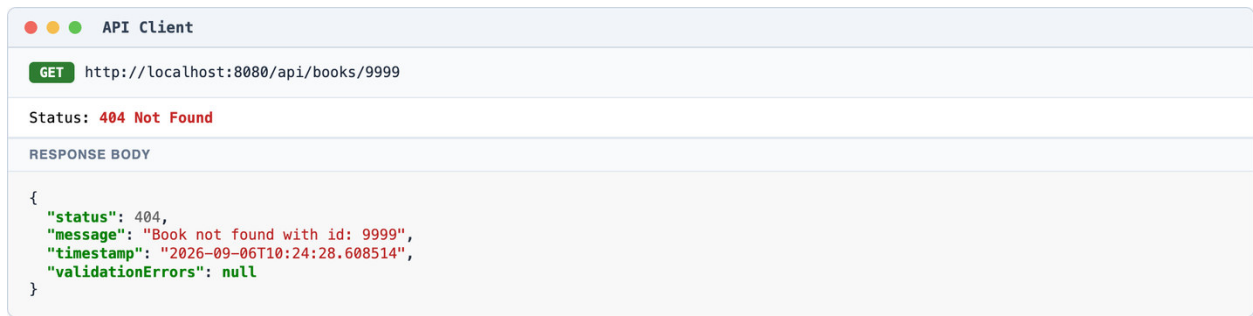


Figure 5.16: Requesting a non-existent book (404 Not Found)

5.4.5. CRUD Operations

Complete CRUD operations are provided for books, authors, categories, members and loans. The controller below is representative: it delegates every operation to the service layer and wraps the result in the standard ApiResponse envelope.

```
BookController.java REST controller

2  @RestController
14  @RequestMapping("/api/books")
15  @RequiredArgsConstructor
16  public class BookController {
17
18      private final BookService bookService;
19
20      @PostMapping
21      public ResponseEntity<ApiResponse<BookDTO>> create(@Valid @RequestBody BookDTO dto) {
22          return ResponseEntity.ok(ApiResponse.success("Book created successfully", bookService.createBook(dto)));
23      }
24
25      @PutMapping("/{id}")
26      public ResponseEntity<ApiResponse<BookDTO>> update(@PathVariable Long id, @Valid @RequestBody BookDTO dto) {
27          return ResponseEntity.ok(ApiResponse.success("Book updated successfully", bookService.updateBook(id, dto)));
28      }
29
30      @DeleteMapping("/{id}")
31      public ResponseEntity<ApiResponse<Void>> delete(@PathVariable Long id) {
32          bookService.deleteBook(id);
33          return ResponseEntity.ok(ApiResponse.success("Book deleted successfully", null));
34      }
35
36      @GetMapping("/{id}")
37      public ResponseEntity<ApiResponse<BookDTO>> getById(@PathVariable Long id) {
38          return ResponseEntity.ok(ApiResponse.success("Book fetched successfully", bookService.getBookById(id)));
39      }
40
41      // Search + filter + sort + pagination:
42      // GET /api/books?title=1984&categoryId=1&authorId=1&availableOnly=true&page=0&size=10&sort=title,asc
43      @GetMapping
44      public ResponseEntity<ApiResponse<Page<BookDTO>>> search(
45          @RequestParam(required = false) String title,
46          @RequestParam(required = false) Long categoryId,
47          @RequestParam(required = false) Long authorId,
48          @RequestParam(required = false) Boolean availableOnly,
49          Pageable pageable) {
50          Page<BookDTO> result = bookService.searchBooks(title, categoryId, authorId, availableOnly, pageable);
51          return ResponseEntity.ok(ApiResponse.success("Books fetched successfully", result));
52      }
53
54      @PostMapping("/{id}/cover")
55      public ResponseEntity<ApiResponse<String>> uploadCover(@PathVariable Long id, @RequestParam("file") MultipartFile file) {
56          String path = bookService.uploadCoverImage(id, file);
57          return ResponseEntity.ok(ApiResponse.success("Cover image uploaded successfully", path));
58      }
59  }
60
61
```

Figure 5.17: BookController.java — REST endpoints for the book resource

```

1  @Service
2  @Repository
3  @Transactional
4  public class BookServiceImpl implements BookService {
5
6      private final BookRepository bookRepository;
7      private final AuthorRepository authorRepository;
8      private final CategoryRepository categoryRepository;
9      private final AuditService auditService;
10
11      private static final String UPLOAD_DIR = "src/main/resources/uploads";
12
13      @Override
14      @Transactional
15      public BookDTO createBook(BookDTO dto) {
16          if (bookRepository.existsById(dto.getId())) {
17              throw new RuntimeException("A book with ISBN " + dto.getId() + " already exists!");
18          }
19          Book book = mapToEntity(dto);
20          Book saved = bookRepository.save(book);
21          auditService.logCurrentEvent(), "CREATE", "Book", "Created book: " + saved.getId();
22          return mapToDto(saved);
23      }
24
25      @Override
26      @Transactional
27      public BookDTO updateBook(Long id, BookDTO dto) {
28          Book book = bookRepository.findById(id);
29          if (book == null) {
30              throw new RuntimeException("Book not found with id: " + id);
31          }
32          book.setId(dto.getId());
33          book.setTitle(dto.getTitle());
34          book.setAuthor(authorRepository.findById(dto.getAuthorId()));
35          book.setCategory(categoryRepository.findById(dto.getCategoryId()));
36          book.setAvailableCopies(dto.getAvailableCopies());
37          book.setCoverImage(dto.getCoverImage());
38          if (dto.getAuthorId() != null) {
39              Author author = authorRepository.findById(dto.getAuthorId());
40              if (author == null) {
41                  throw new RuntimeException("Author not found");
42              }
43              book.setAuthor(author);
44          }
45          if (dto.getCategoryId() != null) {
46              Category category = categoryRepository.findById(dto.getCategoryId());
47              if (category == null) {
48                  throw new RuntimeException("Category not found");
49              }
50              book.setCategory(category);
51          }
52          Book updated = bookRepository.save(book);
53          auditService.logCurrentEvent(), "UPDATE", "Book", "Updated book id: " + id;
54          return mapToDto(updated);
55      }
56
57      @Override
58      @Transactional
59      public void deleteBook(Long id) {
60          Book book = bookRepository.findById(id);
61          if (book == null) {
62              throw new RuntimeException("Book not found with id: " + id);
63          }
64          bookRepository.delete(book);
65          auditService.logCurrentEvent(), "DELETE", "Book", "Deleted book id: " + id;
66      }
67
68      @Override
69      @Transactional(readOnly = true)
70      public BookDTO getBook(Long id) {
71          Book book = bookRepository.findById(id);
72          if (book == null) {
73              throw new RuntimeException("Book not found with id: " + id);
74          }
75          return mapToDto(book);
76      }
77
78      @Override
79      @Transactional(readOnly = true)
80      public Page searchBooks(String title, Long categoryId, Long authorId, Boolean available, Pageable pageable) {
81          Specification<Book> spec = Specification.where(BookSpecification.hasTitle(title));
82          if (categoryId != null) {
83              spec = spec.and(BookSpecification.hasCategoryId(categoryId));
84          }
85          if (authorId != null) {
86              spec = spec.and(BookSpecification.hasAuthorId(authorId));
87          }
88          if (available != null) {
89              spec = spec.and(BookSpecification.isAvailable(available));
90          }
91          return bookRepository.findAll(spec, pageable).map(this::mapToDto);
92      }
93
94      @Override
95      public String uploadCoverImage(Long bookId, MultipartFile file) {
96          Book book = bookRepository.findById(bookId);
97          if (book == null) {
98              throw new RuntimeException("Book not found with id: " + bookId);
99          }
100          try {
101              Path uploadPath = Paths.get(UPLOAD_DIR);
102              if (!Files.exists(uploadPath)) {
103                  Files.createDirectories(uploadPath);
104              }
105              String filename = UUID.randomUUID().toString() + ".jpg";
106              Path filePath = uploadPath.resolve(filename);
107              Files.copy(file.getInputStream(), filePath);
108              book.setCoverImage(filePath.toString());
109              bookRepository.save(book);
110              auditService.logCurrentEvent(), "UPLOAD", "Book", "Uploaded cover for book id: " + bookId;
111          } catch (IOException e) {
112              throw new RuntimeException("Failed to store file: " + e.getMessage());
113          }
114      }
115
116      private String currentUser() {
117          try {
118              return SecurityContextHolder.getContext().getAuthentication().getName();
119          } catch (Exception e) {
120              return "system";
121          }
122      }
123
124      private Book mapToEntity(BookDTO dto) {
125          Book.Builder builder = Book.builder()
126              .id(dto.getId())
127              .title(dto.getTitle())
128              .author(authorRepository.findById(dto.getAuthorId()))
129              .category(categoryRepository.findById(dto.getCategoryId()))
130              .availableCopies(dto.getAvailableCopies())
131              .description(dto.getDescription());
132          if (dto.getAuthorId() != null) {
133              Author author = authorRepository.findById(dto.getAuthorId());
134              if (author == null) {
135                  throw new RuntimeException("Author not found with id: " + dto.getAuthorId());
136              }
137              builder.author(author);
138          }
139          if (dto.getCategoryId() != null) {
140              Category category = categoryRepository.findById(dto.getCategoryId());
141              if (category == null) {
142                  throw new RuntimeException("Category not found with id: " + dto.getCategoryId());
143              }
144              builder.category(category);
145          }
146          return builder.build();
147      }
148
149      private BookDTO mapToDto(Book book) {
150          return BookDTO.builder()
151              .id(book.getId())
152              .title(book.getTitle())
153              .author(book.getAuthor().getId() != null ? book.getAuthor().getId() : null)
154              .authorName(book.getAuthor() != null ? book.getAuthor().getName() : null)
155              .categoryId(book.getCategoryId() != null ? book.getCategoryId().getId() : null)
156              .categoryName(book.getCategoryId() != null ? book.getCategoryId().getName() : null)
157              .totalCopies(book.getTotalCopies())
158              .availableCopies(book.getAvailableCopies())
159              .coverImage(book.getCoverImage())
160              .description(book.getDescription())
161              .build();
162      }
163
164      }
165
166

```

Figure 5.18: BookServiceImpl.java — business logic and entity/DTO mapping

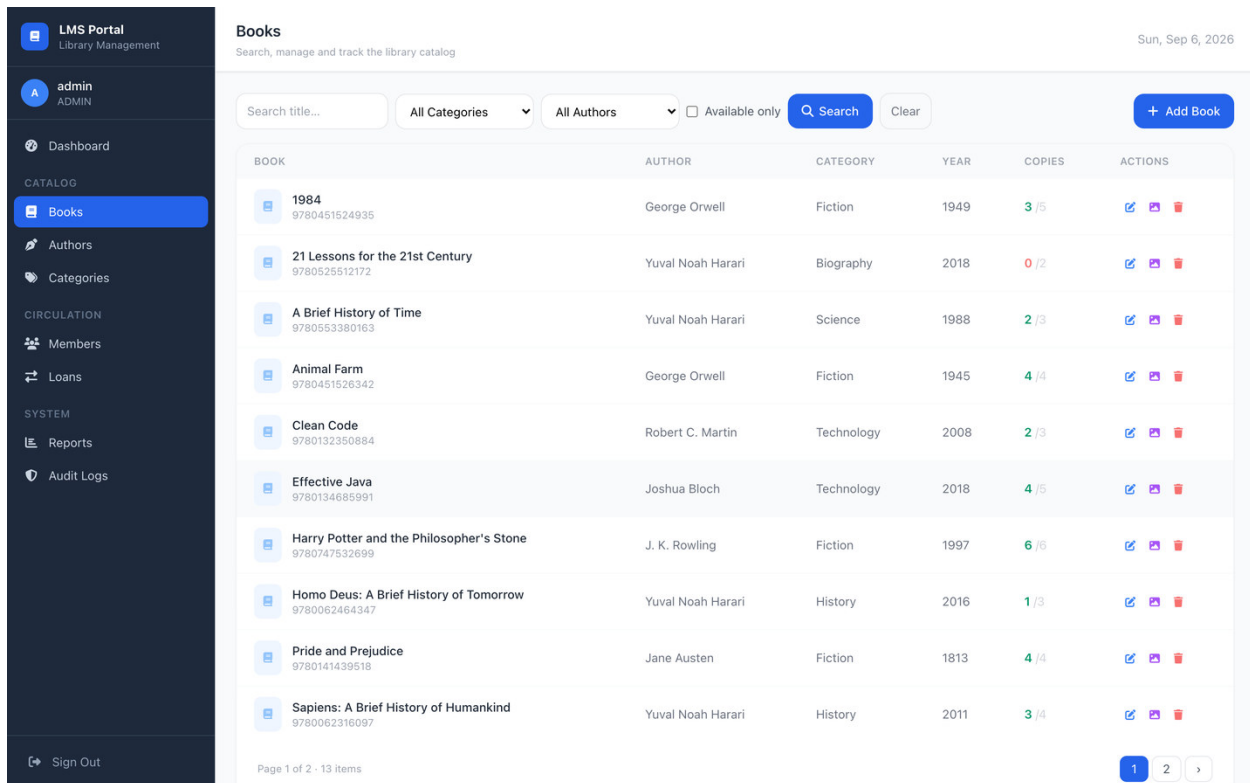


Figure 5.19: Book catalogue in the web client

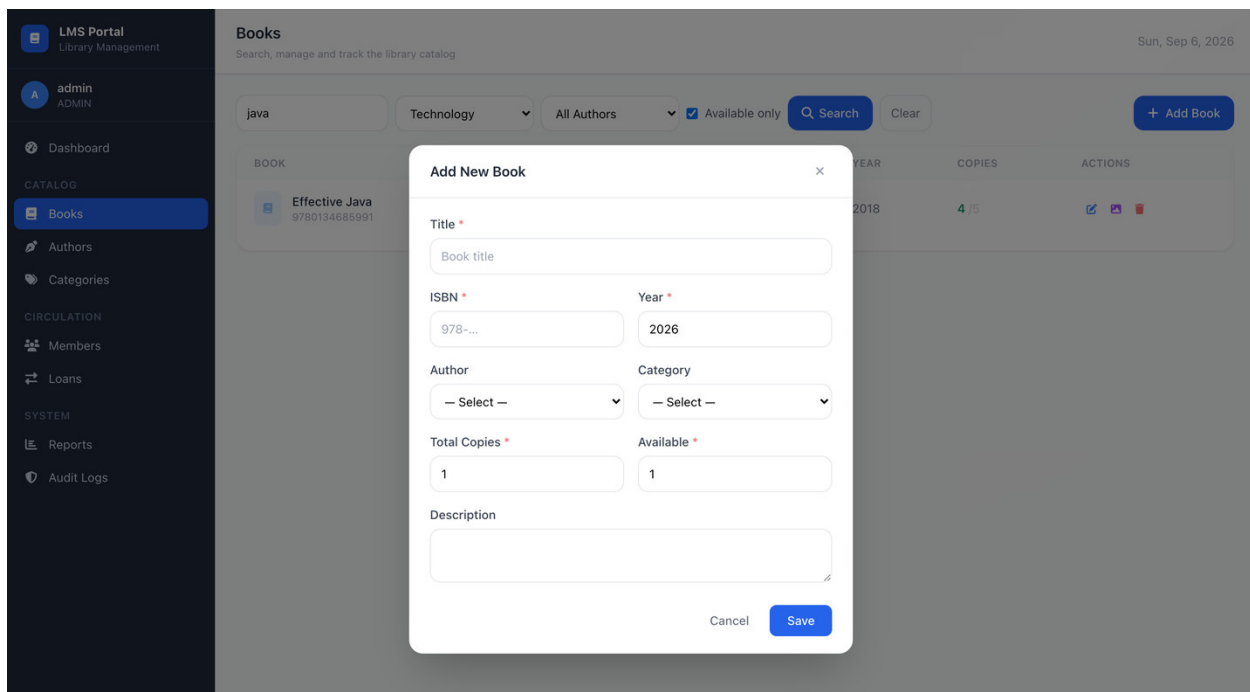


Figure 5.20: Creating a new book record

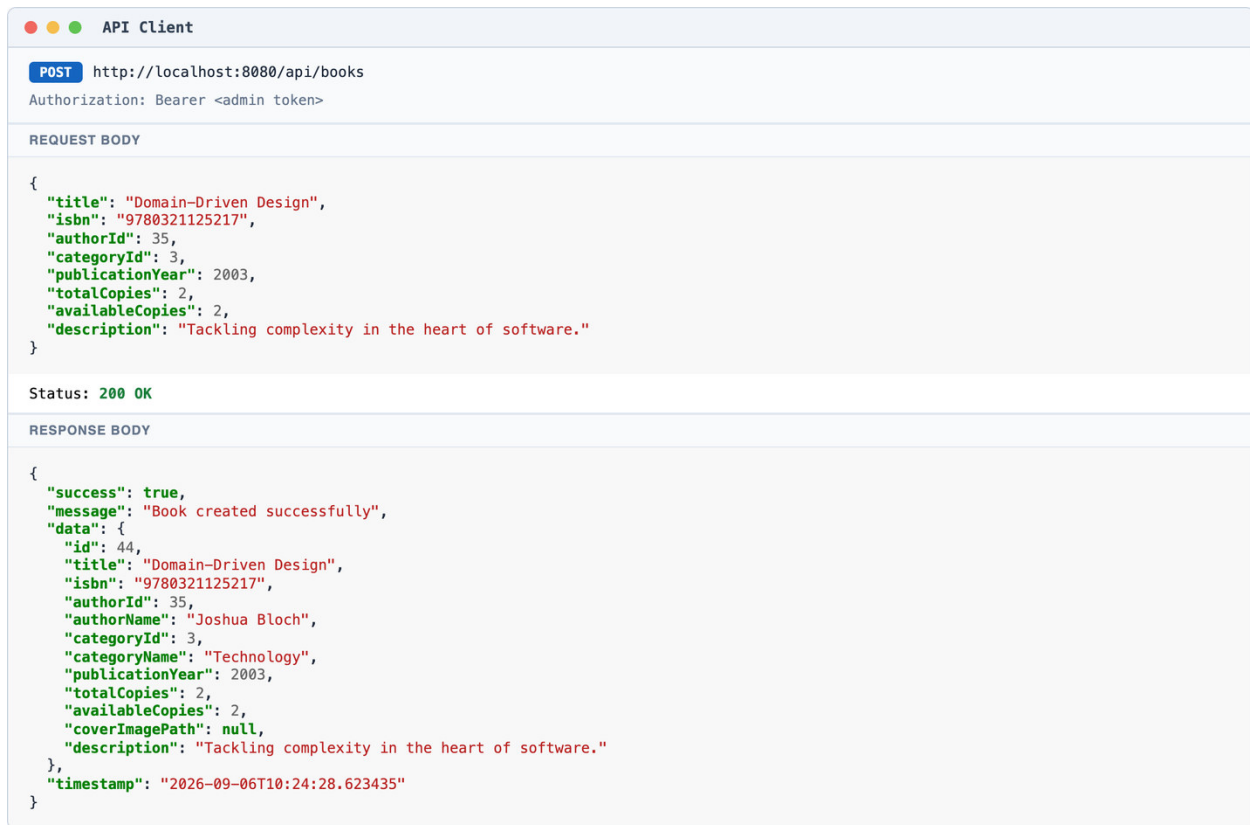


Figure 5.21: Creating a book (POST /api/books)

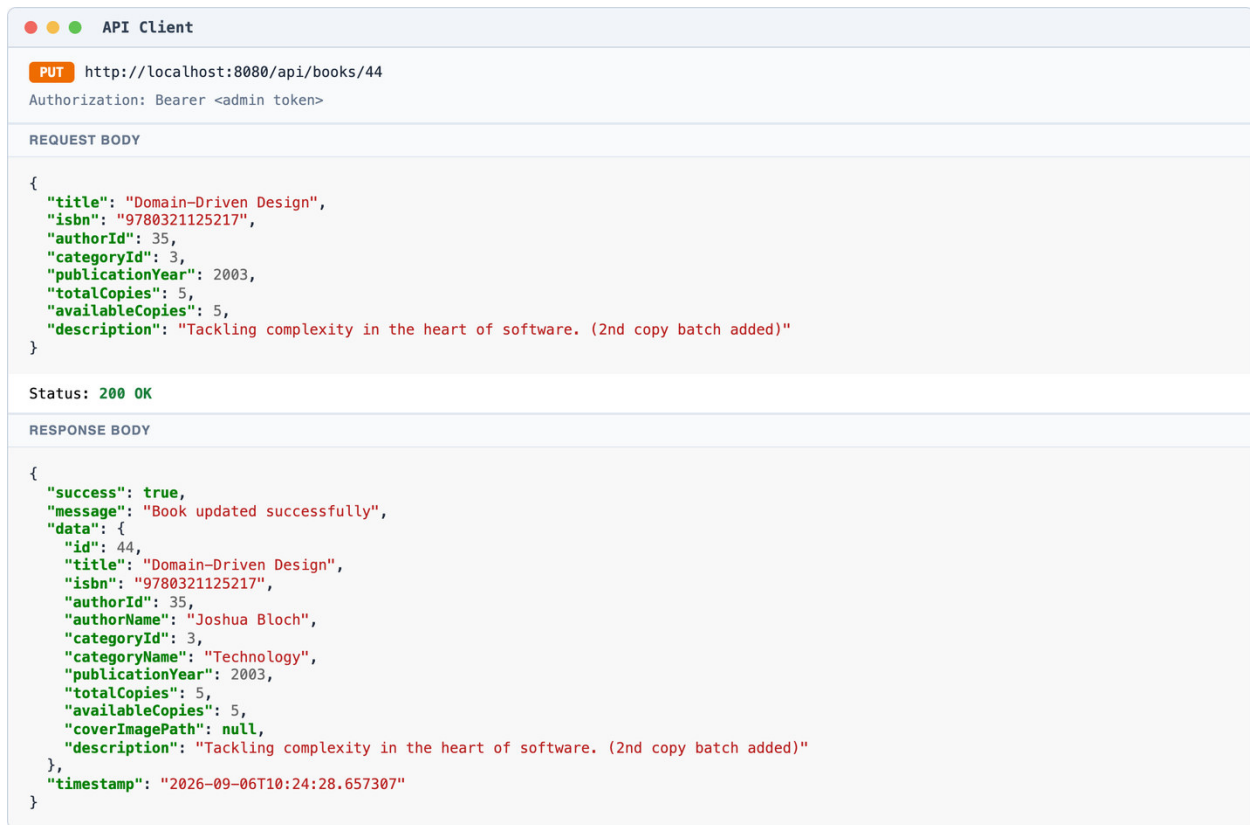


Figure 5.22: Updating a book (PUT /api/books/{id})

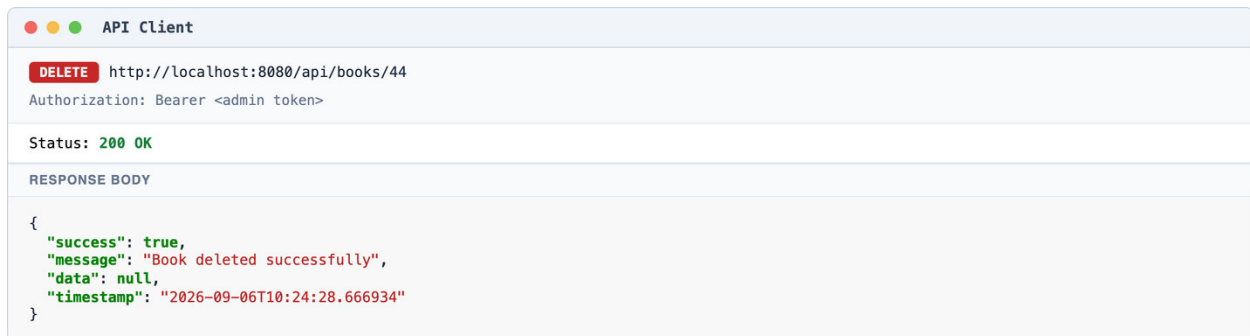


Figure 5.23: Deleting a book (DELETE /api/books/{id})



Figure 5.24: Retrieving a single book (GET /api/books/{id})

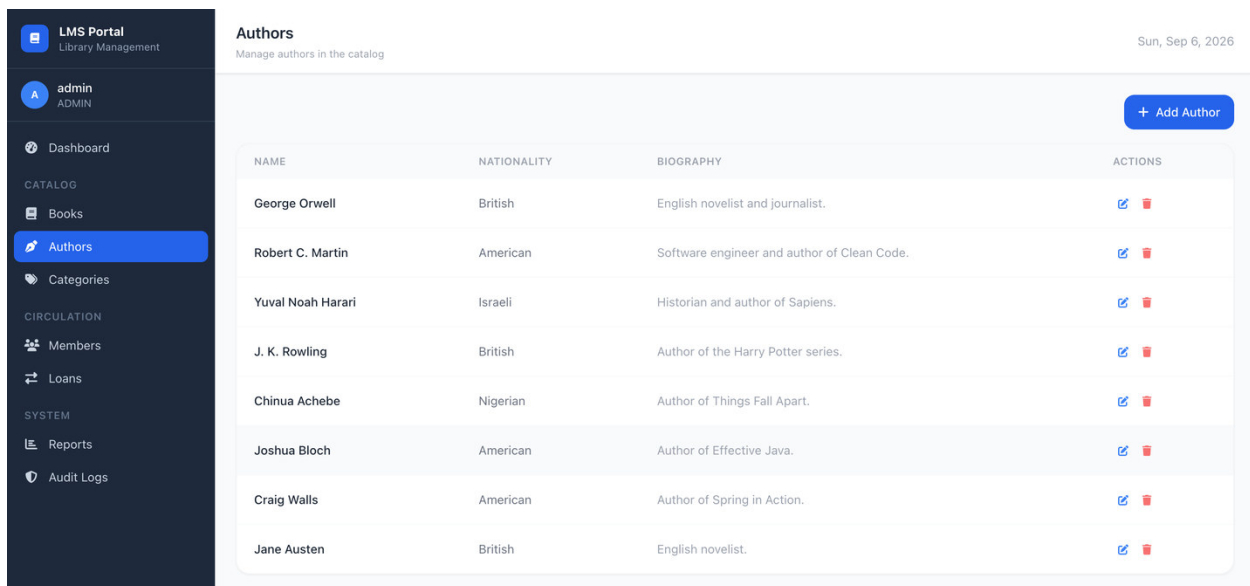


Figure 5.25: Author management

LMS Portal
Library Management

admin
ADMIN

Dashboard

CATALOG

Books

Authors

Categories

CIRCULATION

Members

Loans

SYSTEM

Reports

Audit Logs

Categories

Manage book genres and categories

Sun, Sep 6, 2026

+ Add Category

NAME	DESCRIPTION	ACTIONS
Fiction	Fictional literature	✎ ✖
Science	Scientific books	✎ ✖
Technology	Tech and programming	✎ ✖
History	Historical books	✎ ✖
Biography	Life stories and memoirs	✎ ✖
Mathematics	Pure and applied mathematics	✎ ✖

Figure 5.26: Category management

LMS Portal
Library Management

admin
ADMIN

Dashboard

CATALOG

Books

Authors

Categories

CIRCULATION

Members

Loans

SYSTEM

Reports

Audit Logs

Members

Manage library member accounts

Sun, Sep 6, 2026

All Status

Q Search

Clear

+ Add Member

MEMBER	EMAIL	PHONE	STATUS	ACTIONS
A Anjali Thapa	anjali.thapa@example.com	9800000003	ACTIVE	✎ ✖
B Bikash Shrestha	bikash.shrestha@example.com	9800000004	ACTIVE	✎ ✖
D Deepak Karki	deepak.karki@example.com	9800000008	ACTIVE	✎ ✖
N Nisha Adhikari	nisha.adhikari@example.com	9800000005	ACTIVE	✎ ✖
P Prakash Rai	prakash.raai@example.com	9800000006	ACTIVE	✎ ✖
R Ram Sharma	ram.sharma@example.com	9800000001	ACTIVE	✎ ✖
S Sita Gurung	sita.gurung@example.com	9800000002	ACTIVE	✎ ✖
S Sunita Bhandari	sunita.bhandari@example.com	9800000007	ACTIVE	✎ ✖

Figure 5.27: Member management

5.4.6. Search, Filter, Sort and Pagination

Rather than defining a separate repository method for every combination of filters, the book repository extends `JpaSpecificationExecutor` and the service composes a `Specification` at runtime from whichever optional parameters were supplied. Each predicate returns a neutral conjunction when its parameter is absent, so filters combine freely. Sorting and pagination are supplied by Spring's `Pageable`, which the controller accepts directly as a method parameter.

```
BookSpecification.java Dynamic search criteria

2 public class BookSpecification {
6
7     public static Specification<Book> hasTitle(String title) {
8         return (root, query, cb) -> title == null ? cb.conjunction() :
9             cb.like(cb.lower(root.get("title")), "%" + title.toLowerCase() + "%");
10    }
11
12    public static Specification<Book> hasCategory(Long categoryId) {
13        return (root, query, cb) -> categoryId == null ? cb.conjunction() :
14            cb.equal(root.get("category").get("id"), categoryId);
15    }
16
17    public static Specification<Book> hasAuthor(Long authorId) {
18        return (root, query, cb) -> authorId == null ? cb.conjunction() :
19            cb.equal(root.get("author").get("id"), authorId);
20    }
21
22    public static Specification<Book> availableOnly(Boolean availableOnly) {
23        return (root, query, cb) -> (availableOnly == null || !availableOnly) ? cb.conjunction() :
24            cb.greaterThan(root.get("availableCopies"), 0);
25    }
26 }
27
28
```

Figure 5.28: `BookSpecification.java` — composable, type-safe search predicates

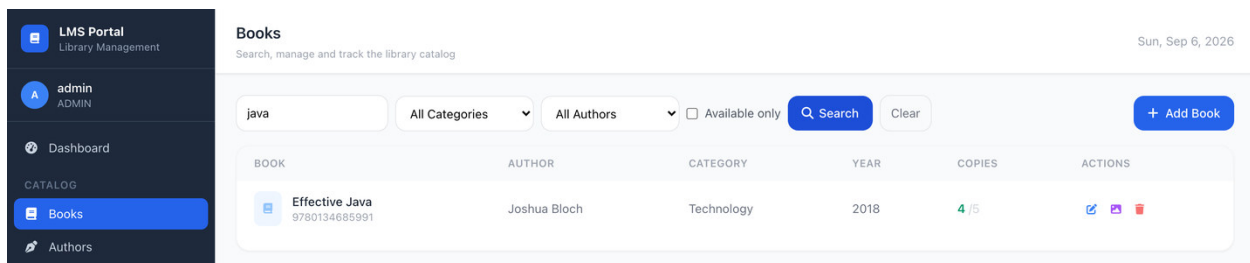


Figure 5.29: Searching the catalogue by title

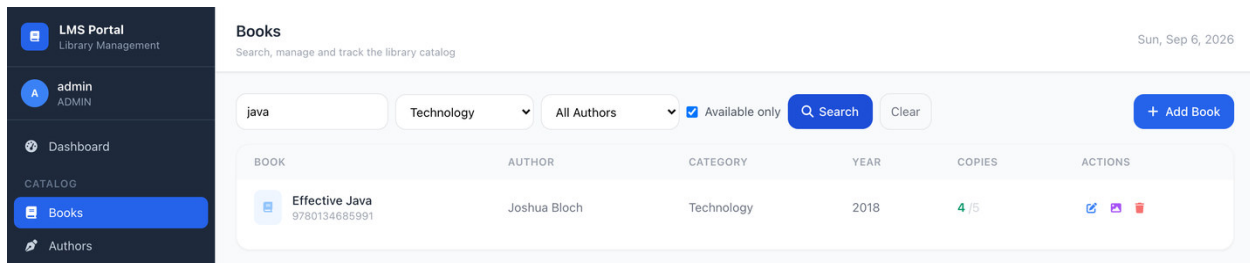


Figure 5.30: Filtering by category and availability

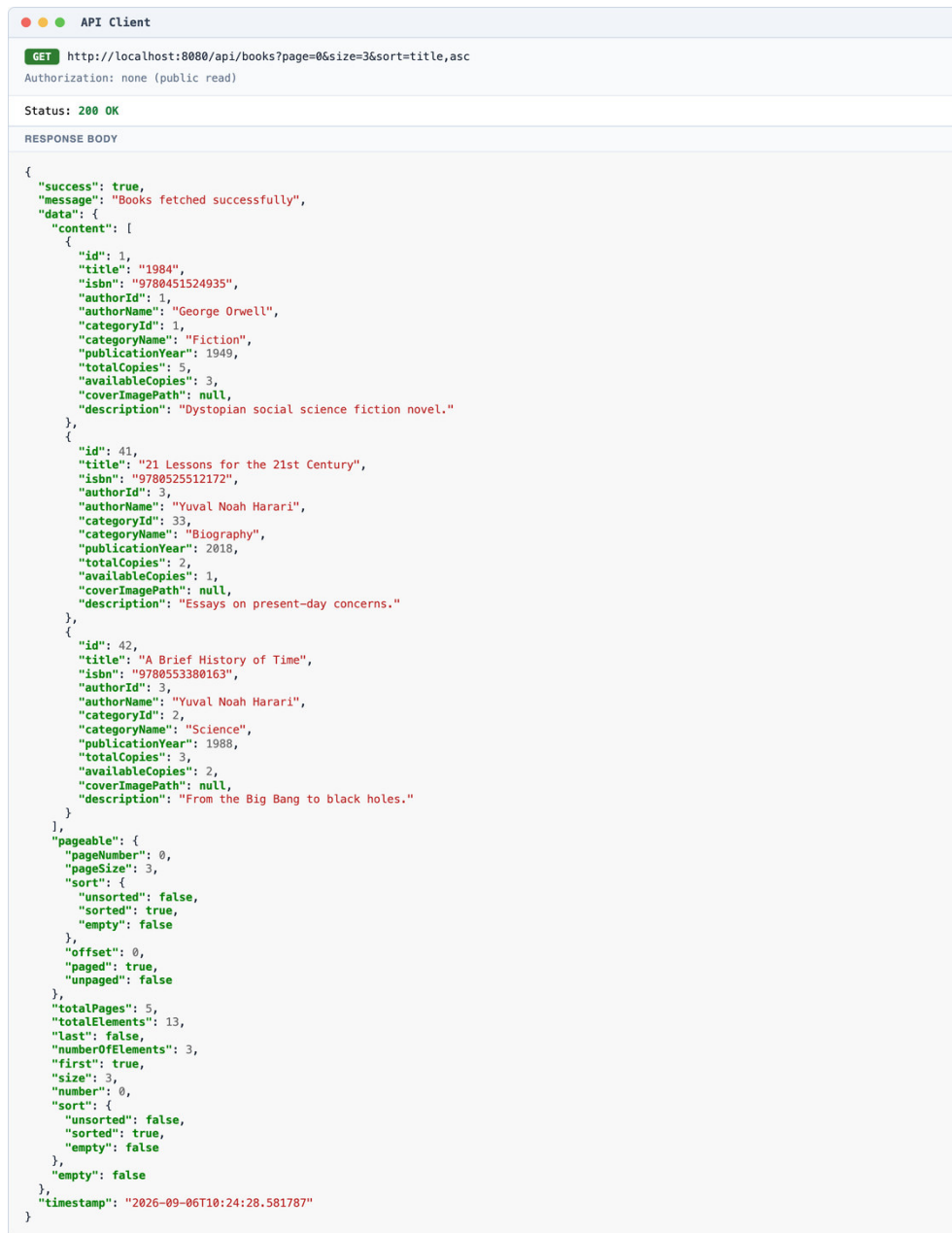


Figure 5.31: Paginated and sorted response with page metadata

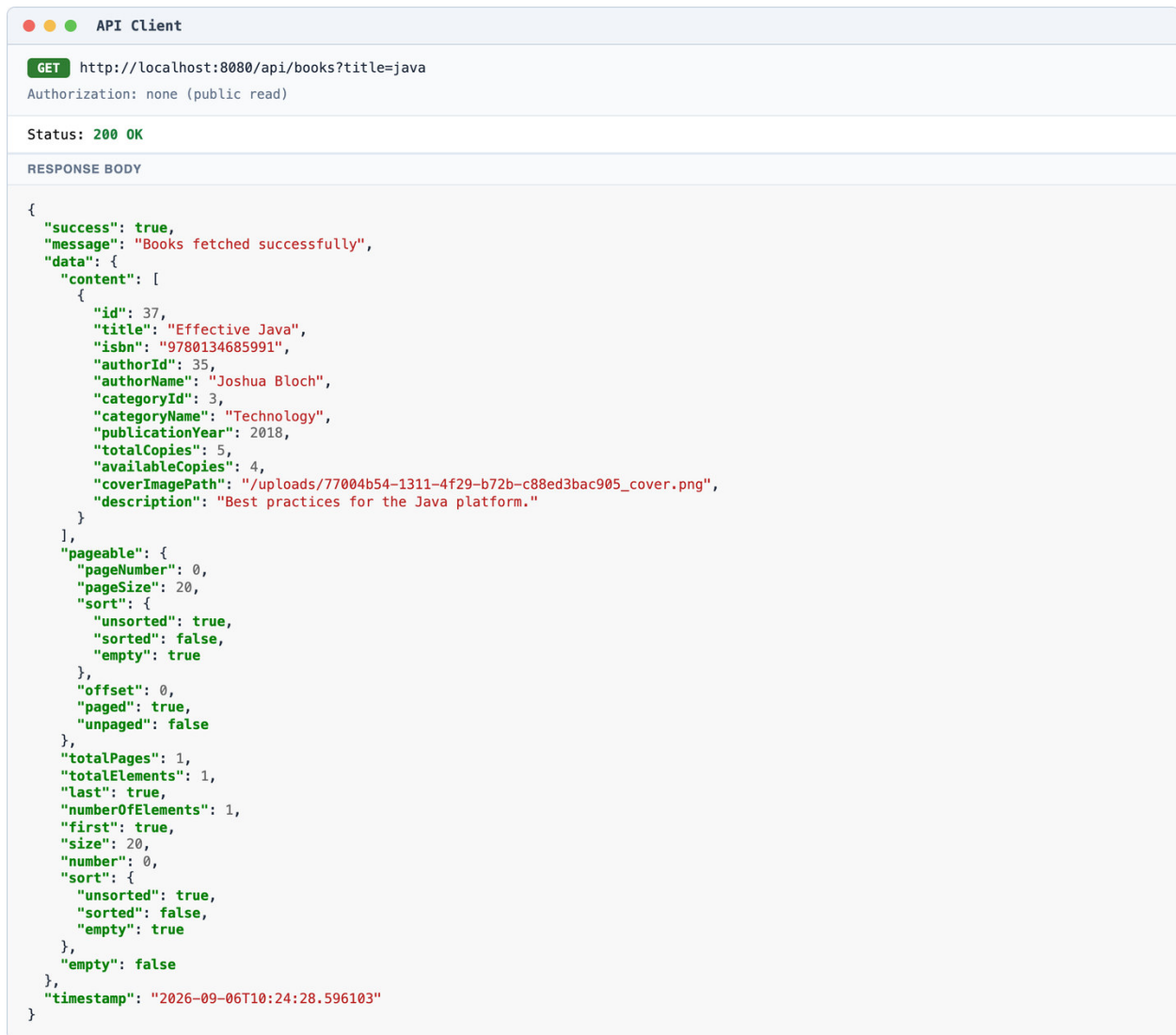


Figure 5.32: Search by title keyword

```
API Client
GET http://localhost:8080/api/books?categoryId=3&availableOnly=true&sort=publicationYear,desc

Status: 200 OK

RESPONSE BODY

{
  "success": true,
  "message": "Books fetched successfully",
  "data": {
    "content": [
      {
        "id": 38,
        "title": "Spring in Action",
        "isbn": "9781617297571",
        "authorId": 36,
        "authorName": "Craig Walls",
        "categoryId": 3,
        "categoryName": "Technology",
        "publicationYear": 2022,
        "totalCopies": 4,
        "availableCopies": 3,
        "coverImagePath": null,
        "description": "A hands-on guide to the Spring Framework."
      },
      {
        "id": 39,
        "title": "The Pragmatic Programmer",
        "isbn": "9780135957059",
        "authorId": 2,
        "authorName": "Robert C. Martin",
        "categoryId": 3,
        "categoryName": "Technology",
        "publicationYear": 2019,
        "totalCopies": 3,
        "availableCopies": 2,
        "coverImagePath": null,
        "description": "From journeyman to master."
      },
      {
        "id": 37,
        "title": "Effective Java",
        "isbn": "9780134685991",
        "authorId": 35,
        "authorName": "Joshua Bloch",
        "categoryId": 3,
        "categoryName": "Technology",
        "publicationYear": 2018,
        "totalCopies": 5,
        "availableCopies": 4,
        "coverImagePath": "/uploads/77004b54-1311-4f29-b72b-c88ed3bac905_cover.png",
        "description": "Best practices for the Java platform."
      },
      {
        "id": 2,
        "title": "Clean Code",
        "isbn": "9780132350884",
        "authorId": 2,
        "authorName": "Robert C. Martin",
        "categoryId": 3,
        "categoryName": "Technology",
        "publicationYear": 2008,
        "totalCopies": 3,
        "availableCopies": 2,
        "coverImagePath": null,
        "description": "A handbook of agile software craftsmanship."
      }
    ],
    "pageable": {
      "pageNumber": 0,
      "pageSize": 20,
      "sort": {
        "unsorted": false,
        "sorted": true,
        "empty": false
      },
      "offset": 0,
      "paged": true,
      "unpaged": false
    },
    "totalPages": 1,
    "totalElements": 4,
    "last": true,
    "numberOfElements": 4,
    "first": true,
    "size": 20,
    "number": 0,
    "sort": {
      "unsorted": false,
      "sorted": true,
      "empty": false
    },
    "empty": false
  },
  "timestamp": "2026-09-06T10:24:28.600961"
}
```

Figure 5.33: Combined category filter, availability filter and descending sort

5.4.7. Loan Issue, Return and Fine Calculation

Issuing a book verifies that a copy is available and that the member is not suspended, decrements the available copy count and creates a loan with a due date fourteen days ahead. Returning a book restores the copy count, records the return date and, if the return is late, calculates a fine of NPR 10 for each overdue day. Both operations are transactional and both write an audit entry.



```
36  @Override
37  @Transactional
38  public LoanDTO issueLoan(LoanDTO dto) {
39      Book book = bookRepository.findById(dto.getBookId())
40          .orElseThrow(() -> new ResourceNotFoundException("Book not found with id: " + dto.getBookId()));
41      Member member = memberRepository.findById(dto.getMemberId())
42          .orElseThrow(() -> new ResourceNotFoundException("Member not found with id: " + dto.getMemberId()));
43
44      if (book.getAvailableCopies() == null || book.getAvailableCopies() <= 0) {
45          throw new BadRequestException("No available copies of book: " + book.getTitle());
46      }
47      if (member.getStatus() == Member.MemberStatus.SUSPENDED) {
48          throw new BadRequestException("Member is suspended and cannot borrow books");
49      }
50
51      book.setAvailableCopies(book.getAvailableCopies() - 1);
52      bookRepository.save(book);
53
54      Loan loan = Loan.builder().book(book).member(member).build();
55      Loan saved = loanRepository.save(loan);
56      auditService.log(currentUser(), "CREATE", "Loan", "Issued book " + book.getTitle() + " to " + member.getFullName());
57      return mapToDTO(saved);
58  }
59
60  @Override
61  @Transactional
62  public LoanDTO returnLoan(Long loanId) {
63      Loan loan = loanRepository.findById(loanId)
64          .orElseThrow(() -> new ResourceNotFoundException("Loan not found with id: " + loanId));
65      if (loan.getStatus() == Loan.LoanStatus.RETURNED) {
66          throw new BadRequestException("This loan has already been returned");
67      }
68      loan.setReturnDate(LocalDate.now());
69      loan.setStatus(Loan.LoanStatus.RETURNED);
70
71      if (loan.getReturnDate().isAfter(loan.getDueDate())) {
72          long overdueDays = ChronoUnit.DAYS.between(loan.getDueDate(), loan.getReturnDate());
73          loan.setFineAmount(overdueDays * FINE_PER_DAY);
74      }
75
76      Book book = loan.getBook();
77      book.setAvailableCopies(book.getAvailableCopies() + 1);
78      bookRepository.save(book);
79
80      Loan updated = loanRepository.save(loan);
81      auditService.log(currentUser(), "UPDATE", "Loan", "Returned loan id: " + loanId);
82      return mapToDTO(updated);
83  }
84
85  @Override
86  @Transactional(readOnly = true)
```

Figure 5.34: LoanServiceImpl.java — issue and return with automatic fine calculation

LMS Portal
Library Management

admin
ADMIN

Dashboard

CATALOG

Books

Authors

Categories

CIRCULATION

Members

Loans

SYSTEM

Reports

Audit Logs

Loans

Issue and manage book loans

Sun, Sep 6, 2026

+ Issue Loan

BOOK	MEMBER	ISSUE	DUE	RETURNED	STATUS	FINE	ACTIONS
21 Lessons for the 21st Century	Prakash Rai	2026-09-06	2026-09-20	—	ISSUED	—	—
Homo Deus: A Brief History of Tomorrow	Prakash Rai	2026-09-06	2026-09-20	—	ISSUED	—	—
21 Lessons for the 21st Century	Prakash Rai	2026-09-06	2026-09-20	—	ISSUED	—	—
Homo Deus: A Brief History of Tomorrow	Prakash Rai	2026-09-06	2026-09-20	—	ISSUED	—	—
A Brief History of Time	Deepak Karki	2026-09-06	2026-09-20	—	ISSUED	—	—
Sapiens: A Brief History of Humankind	Sunita Bhandari	2026-09-06	2026-09-20	—	ISSUED	—	—
The Pragmatic Programmer	Prakash Rai	2026-09-06	2026-09-20	—	ISSUED	—	—
Harry Potter and the Philosopher's Stone	Nisha Adhikari	2026-08-17	2026-08-31	2026-09-06	RETURNED	\$60.00	—
Spring in Action	Bikash Shrestha	2026-09-06	2026-09-20	—	ISSUED	—	—
Effective Java	Anjali Thapa	2026-08-12	2026-08-26	—	OVERDUE	—	Return

Page 1 of 2 · 13 items

1 2 >

Figure 5.35: Loan register showing issued, returned and overdue loans

API Client

POST

http://localhost:8080/api/loans/issue

Authorization: Bearer <admin token>

REQUEST BODY

```
{
  "bookId": 40,
  "memberId": 36
}
```

Status: 200 OK

RESPONSE BODY

```
{
  "success": true,
  "message": "Book issued successfully",
  "data": {
    "id": 12,
    "bookId": 40,
    "bookTitle": "Homo Deus: A Brief History of Tomorrow",
    "memberId": 36,
    "memberName": "Prakash Rai",
    "issueDate": "2026-09-06",
    "dueDate": "2026-09-20",
    "returnDate": null,
    "status": "ISSUED",
    "fineAmount": 0.0
  },
  "timestamp": "2026-09-06T10:24:28.686639"
}
```

Figure 5.36: Issuing a book to a member

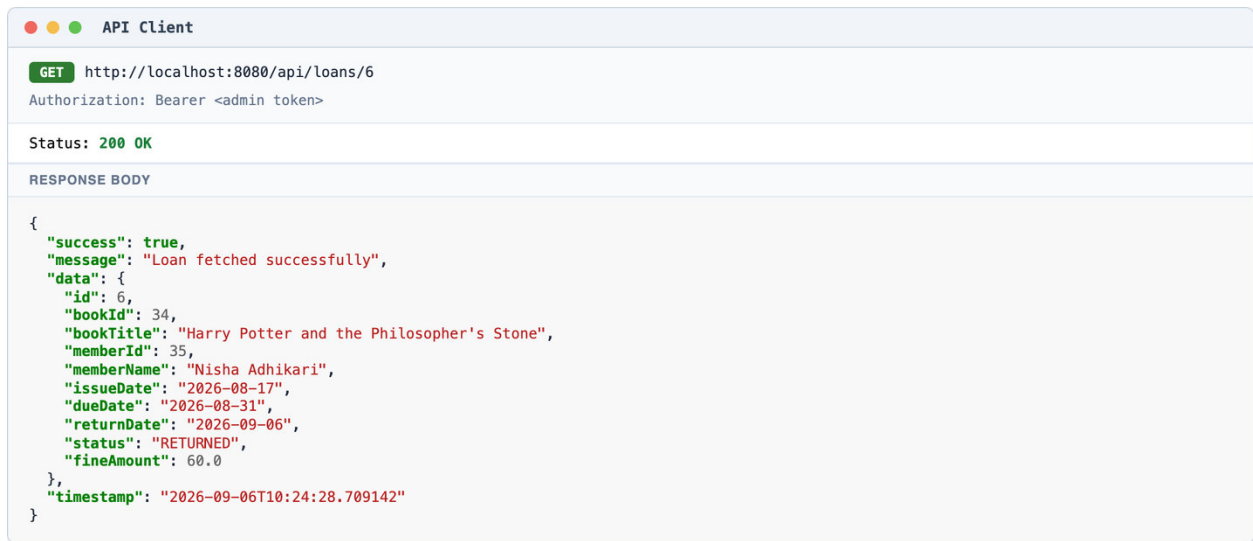


Figure 5.37: A loan returned six days late, with a fine of NPR 60 calculated automatically

5.4.8. File Upload and Download

The first additional feature is book cover image upload. The endpoint accepts a multipart file, stores it under a generated unique filename to avoid collisions, records the resulting path on the book record and returns that path. WebConfig maps the upload directory as a static resource so the stored image can then be served directly, and the maximum upload size is capped at 5 MB in configuration.



Figure 5.38: Uploading a book cover image as multipart form-data

5.4.9. Scheduled Task and E-mail Notification

The second additional feature is a scheduled background job. Every day at 08:00 the scheduler queries for loans whose due date has passed and whose status is still ISSUED, marks each one OVERDUE and dispatches a reminder e-mail to the member concerned. When SMTP is disabled the message is written to the application log instead, so the behaviour can be demonstrated without external mail credentials. The method is transactional so that the member and book associated with each loan can be read while the persistence context is still open.



```
1  /**
2  17  * Scheduled task that runs daily to detect overdue loans,
3  18  * mark them as OVERDUE, and send an email notification (if enabled)
4  19  * to remind members to return their books.
5  20  */
6  21  @Component
7  22  @RequiredArgsConstructor
8  23  @Slf4j
9  24  public class OverdueLoanScheduler {
10  25
11  26      private final LoanRepository loanRepository;
12  27      private final JavaMailSender mailSender;
13  28
14  29      @Value("${app.mail.enabled:false}")
15  30      private boolean mailEnabled;
16  31
17  32      // Runs every day at 8:00 AM
18  33      @Scheduled(cron = "0 0 8 * * *")
19  34      @Transactional
20  35      public void checkOverdueLoans() {
21  36          List<Loan> overdue = loanRepository.findOverdueLoans(LocalDate.now());
22  37          for (Loan loan : overdue) {
23  38              loan.setStatus(Loan.LoanStatus.OVERDUE);
24  39              loanRepository.save(loan);
25  40              notifyMember(loan);
26  41          }
27  42          log.info("Overdue loan check completed. {} loan(s) marked overdue.", overdue.size());
28  43      }
29  44
30  45      private void notifyMember(Loan loan) {
31  46          if (!mailEnabled) {
32  47              log.info("[Email Simulation] Reminder: {} - Book '{}' was due on {}",
33  48                  loan.getMember().getEmail(), loan.getBook().getTitle(), loan.getDueDate());
34  49              return;
35  50          }
36  51          try {
37  52              SimpleMailMessage message = new SimpleMailMessage();
38  53              message.setTo(loan.getMember().getEmail());
39  54              message.setSubject("Library Book Overdue Reminder");
40  55              message.setText("Dear " + loan.getMember().getFullName() +
41  56                  "\n\nThe book \"" + loan.getBook().getTitle() + "\" was due on " + loan.getDueDate() +
42  57                  ". Please return it as soon as possible to avoid additional fines.\n\nThank you,\nLibrary Management System");
43  58              mailSender.send(message);
44  59          } catch (Exception e) {
45  60              log.error("Failed to send overdue email to {}: {}", loan.getMember().getEmail(), e.getMessage());
46  61          }
47  62      }
48  63  }
49  64
50  65  }
```

Figure 5.39: OverdueLoanScheduler.java — daily cron job with e-mail notification

```

LoanRepository.java
Spring Data JPA repository

2  public interface LoanRepository extends JpaRepository<Loan, Long>, JpaSpecificationExecutor<Loan> {
11
12      @Query("SELECT l FROM Loan l WHERE l.status = 'ISSUED' AND l.dueDate < :today")
13      List<Loan> findOverdueLoans(LocalDate today);
14
15      List<Loan> findByMemberId(Long memberId);
16  }
17
18

```

Figure 5.40: LoanRepository.java — the JPQL query that identifies overdue loans

```

logs/lms-application.log - overdue scheduler

2026-09-06T10:22:20.038+05:45 INFO 41272 --- [ scheduling-1] c.l.lms.scheduler.OverdueLoanScheduler : [Email Simulation] Reminder: ram.sharma@example.com - Book '1984' was due on 2026-09-06T10:22:20.038+05:45
2026-09-06T10:22:20.039+05:45 INFO 41272 --- [ scheduling-1] c.l.lms.scheduler.OverdueLoanScheduler : [Email Simulation] Reminder: anjali.thapa@example.com - Book 'Effective Java' was due on 2026-09-06T10:22:20.039+05:45
2026-09-06T10:22:20.040+05:45 INFO 41272 --- [ scheduling-1] c.l.lms.scheduler.OverdueLoanScheduler : [Email Simulation] Reminder: nisha.adhikari@example.com - Book 'Harry Potter and the Chamber of Secrets' was due on 2026-09-06T10:22:20.040+05:45
2026-09-06T10:22:20.040+05:45 INFO 41272 --- [ scheduling-1] c.l.lms.scheduler.OverdueLoanScheduler : Overdue loan check completed. 3 loan(s) marked overdue.

```

Figure 5.41: Application log showing three overdue loans detected and three reminders dispatched

5.4.10. PDF and CSV Report Generation

The third and fourth additional features are exportable reports. The circulation report is generated as a formatted PDF using iText, and the inventory report as a CSV file using OpenCSV. Both are produced in memory and streamed to the client as file attachments, and both are restricted to ADMIN and LIBRARIAN roles.

```
ReportServiceImpl.java iText PDF report generation

26  @Override
27  @Transactional(readOnly = true)
28  public ByteArrayOutputStream generateLoansPdfReport() {
29      ByteArrayOutputStream out = new ByteArrayOutputStream();
30      try {
31          Document document = new Document();
32          PdfWriter.getInstance(document, out);
33          document.open();
34
35          Font titleFont = FontFactory.getFont(FontFactory.HELVETICA_BOLD, 18);
36          Paragraph title = new Paragraph("Library Management System - Loans Report", titleFont);
37          title.setAlignment(Element.ALIGN_CENTER);
38          document.add(title);
39          document.add(new Paragraph(" "));
40
41          PdfPTable table = new PdfPTable(6);
42          table.setWidthPercentage(100);
43          String[] headers = {"Loan ID", "Book", "Member", "Issue Date", "Due Date", "Status"};
44          for (String h : headers) {
45              PdfPCell cell = new PdfPCell(new Phrase(h, FontFactory.getFont(FontFactory.HELVETICA_BOLD, 11)));
46              table.addCell(cell);
47          }
48
49          List<Loan> loans = loanRepository.findAll();
50          for (Loan loan : loans) {
51              table.addCell(String.valueOf(loan.getId()));
52              table.addCell(loan.getBook().getTitle());
53              table.addCell(loan.getMember().getFullName());
54              table.addCell(String.valueOf(loan.getIssueDate()));
55              table.addCell(String.valueOf(loan.getDueDate()));
56              table.addCell(loan.getStatus().name());
57          }
58          document.add(table);
59          document.close();
60      } catch (DocumentException e) {
61          throw new RuntimeException("Failed to generate PDF report: " + e.getMessage());
62      }
```

Figure 5.42: ReportServiceImpl.java — PDF generation with iText

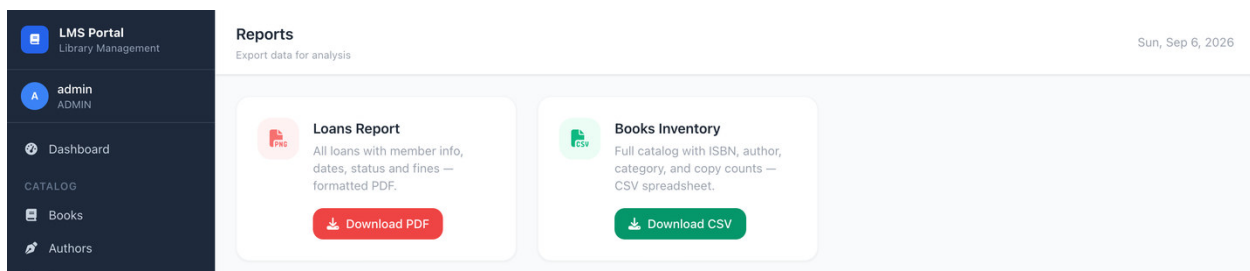


Figure 5.43: Report download page in the web client

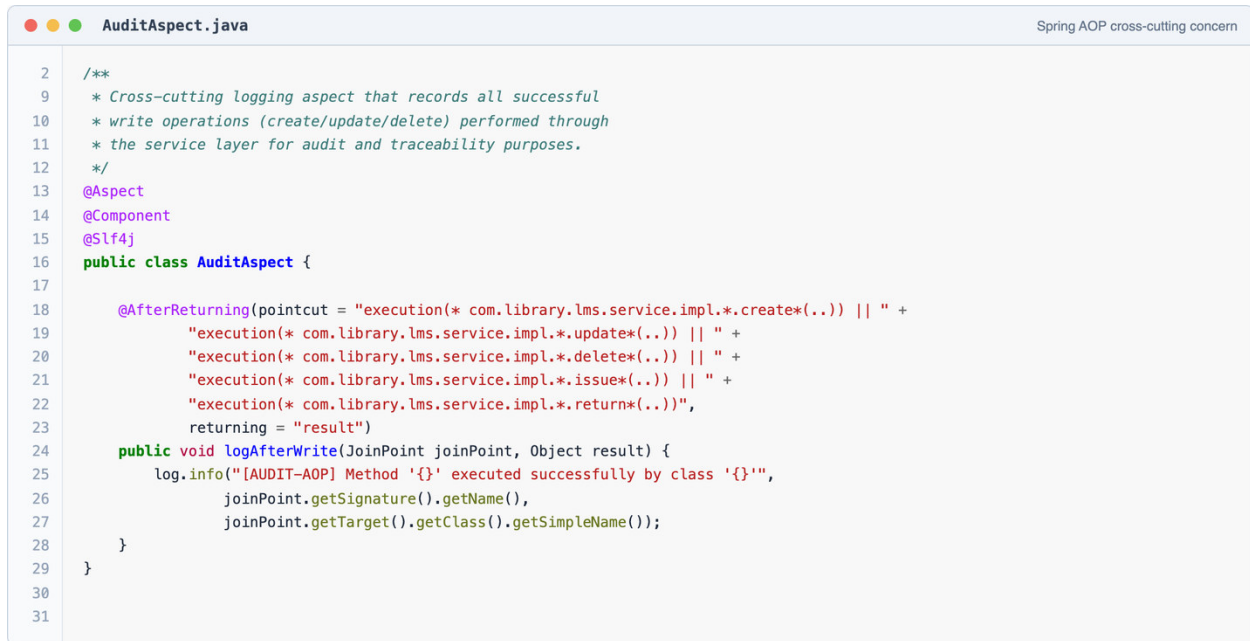
```
books_report.csv – generated by OpenCSV

"ID","Title","ISBN","Author","Category","Total Copies","Available Copies"
"1","1984","9780451524935","George Orwell","Fiction","5","3"
"2","Clean Code","9780132350884","Robert C. Martin","Technology","3","2"
"3","Sapiens: A Brief History of Humankind","9780062316097","Yuval Noah Harari","History","4","3"
"33","Animal Farm","9780451526342","George Orwell","Fiction","4","4"
"34","Harry Potter and the Philosopher's Stone","9780747532699","J. K. Rowling","Fiction","6","6"
"35","Things Fall Apart","9780385474542","Chinua Achebe","Fiction","3","3"
"36","Pride and Prejudice","9780141439518","Jane Austen","Fiction","4","4"
"37","Effective Java","9780134685991","Joshua Bloch","Technology","5","4"
"38","Spring in Action","9781617297571","Craig Walls","Technology","4","3"
"39","The Pragmatic Programmer","9780135957059","Robert C. Martin","Technology","3","2"
"40","Homo Deus: A Brief History of Tomorrow","9780062464347","Yuval Noah Harari","History","3","1"
... (remaining rows omitted)
```

Figure 5.44: Inventory report generated as CSV by OpenCSV

5.4.11. Audit Logging

The fifth additional feature is a persistent audit trail. Every create, update, delete and login operation is written to the `audit_log` table by the `AuditService`, recording the acting username, the action, the entity affected, a human-readable description and a timestamp. This is reinforced by a Spring AOP aspect that intercepts every write method in the service layer and logs it, demonstrating a genuine cross-cutting concern implemented without modifying the business logic it observes. The audit endpoint itself is restricted to administrators.



```
2  /**
9  * Cross-cutting logging aspect that records all successful
10 * write operations (create/update/delete) performed through
11 * the service layer for audit and traceability purposes.
12 */
13 @Aspect
14 @Component
15 @Slf4j
16 public class AuditAspect {
17
18     @AfterReturning(pointcut = "execution(* com.library.lms.service.impl.*.create*(..)) || " +
19         "execution(* com.library.lms.service.impl.*.update*(..)) || " +
20         "execution(* com.library.lms.service.impl.*.delete*(..)) || " +
21         "execution(* com.library.lms.service.impl.*.issue*(..)) || " +
22         "execution(* com.library.lms.service.impl.*.return*(..))",
23         returning = "result")
24     public void logAfterWrite(JoinPoint joinPoint, Object result) {
25         log.info("[AUDIT-AOP] Method '{method}' executed successfully by class '{class}'",
26             joinPoint.getSignature().getName(),
27             joinPoint.getTarget().getClass().getSimpleName());
28     }
29 }
30
31
```

Figure 5.45: `AuditAspect.java` — Spring AOP advice intercepting service-layer write operations

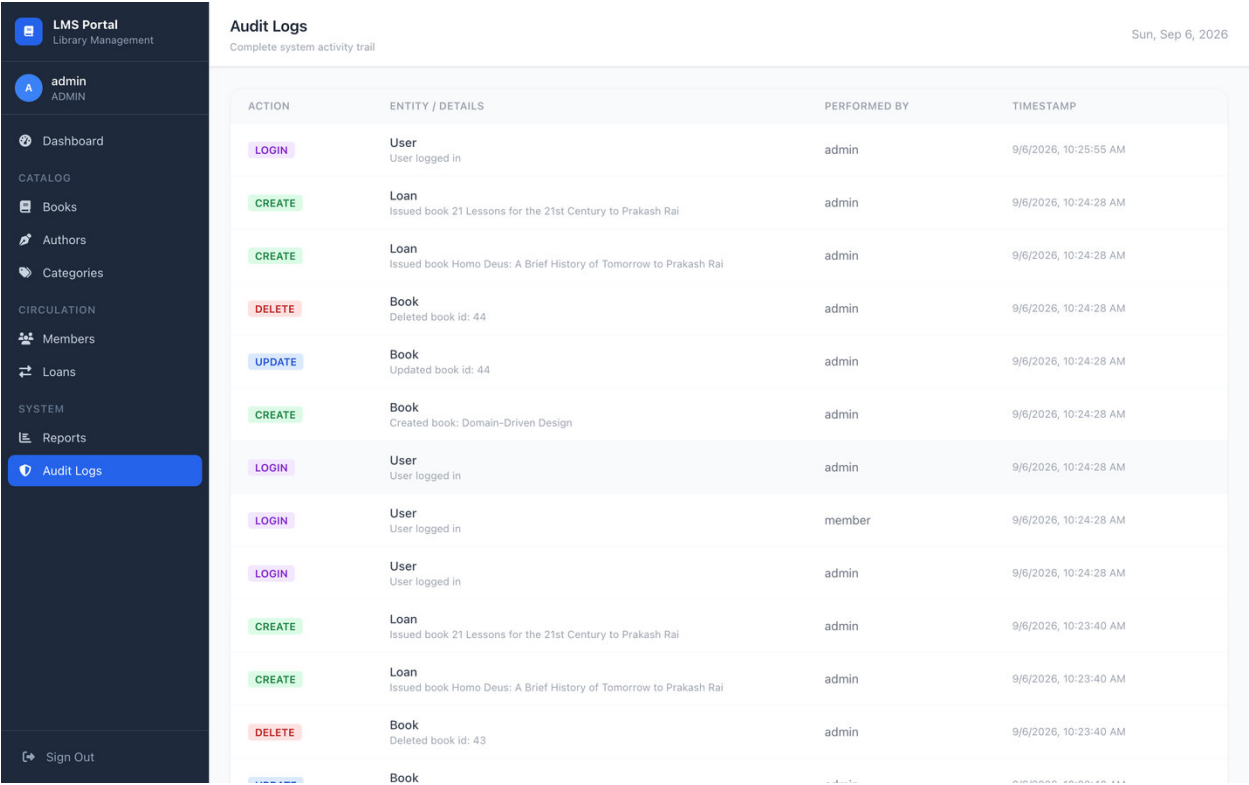


Figure 5.46: Audit trail in the web client

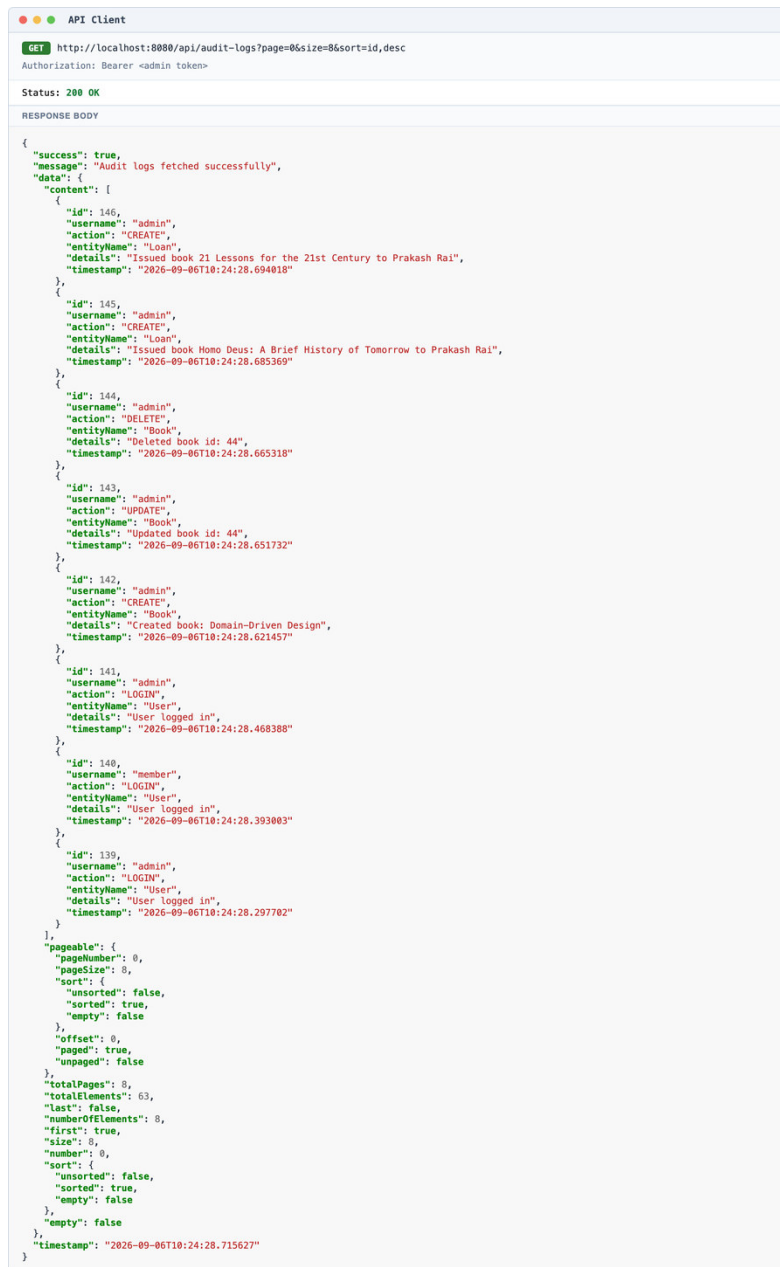


Figure 5.47: Audit trail retrieved through the API

5.4.12. API Documentation and Database Console

SpringDoc OpenAPI publishes an interactive Swagger UI describing every endpoint, its parameters, its request and response schemas and its security requirements. The H2 console provides direct access to the generated schema and the stored rows, which was used throughout development to verify that entities, relationships and constraints were mapped as intended.

Swagger
powered by SMARTBEAR

/v3/api-docs Explore

Library Management System API 1.0.0 OAS 3.0

[/v3/api-docs](#)

REST API for the Library Management System.

MCS-313 — Advanced Java Programming — Final Assignment

How to authenticate

1. Use **POST /api/auth/login** with your credentials
2. Copy the **token** from the response
3. Click **Authorize** above and enter: **Bearer <your-token>**

Default credentials

Username	Password	Role
admin	Admin@123	ADMIN
librarian	Lib@1234	LIBRARIAN
member	Member@123	MEMBER

[Contact MCS-313 Final Assignment](#)

Servers

Authorize

member-controller

- GET** /api/members/{id} lock icon
- PUT** /api/members/{id} lock icon
- DELETE** /api/members/{id} lock icon
- GET** /api/members lock icon

Figure 5.48: Swagger UI — API overview and authentication guide

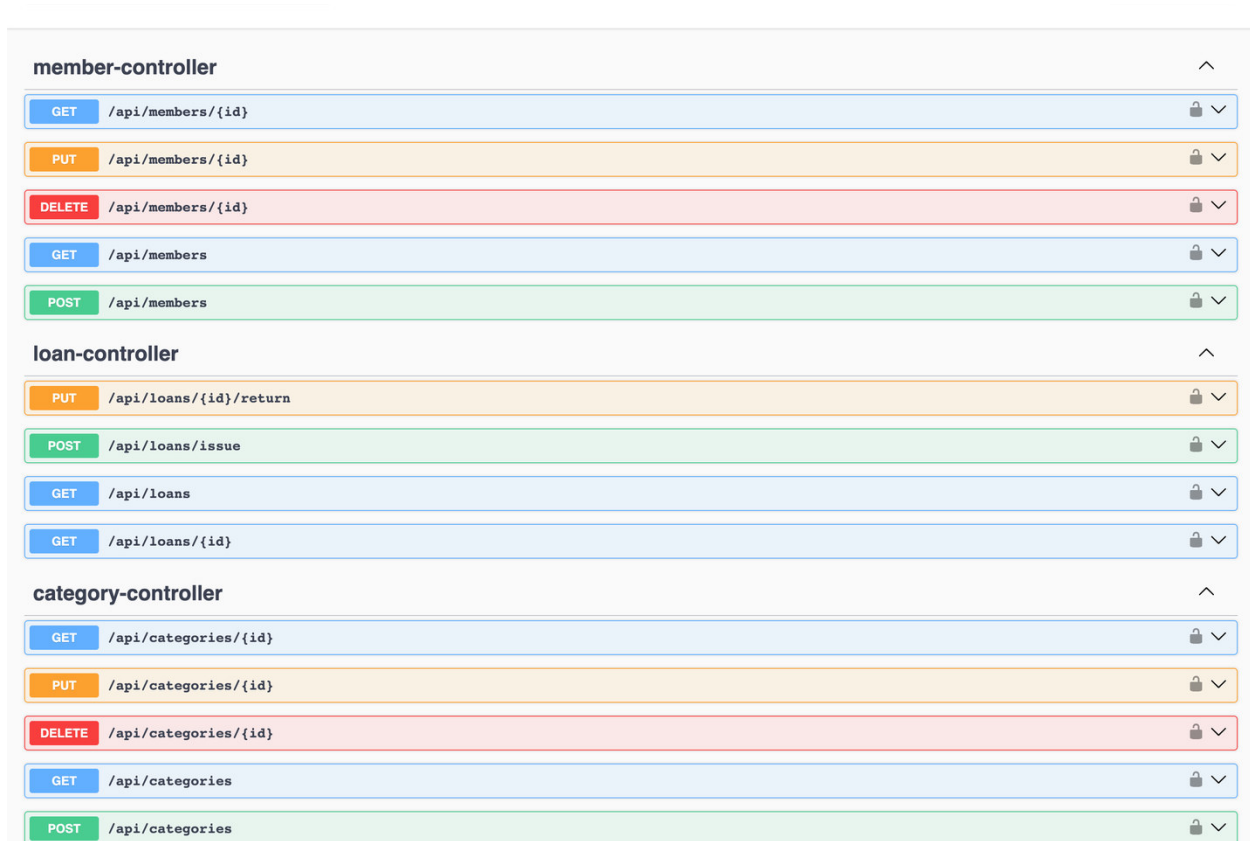


Figure 5.49: Swagger UI — documented endpoints

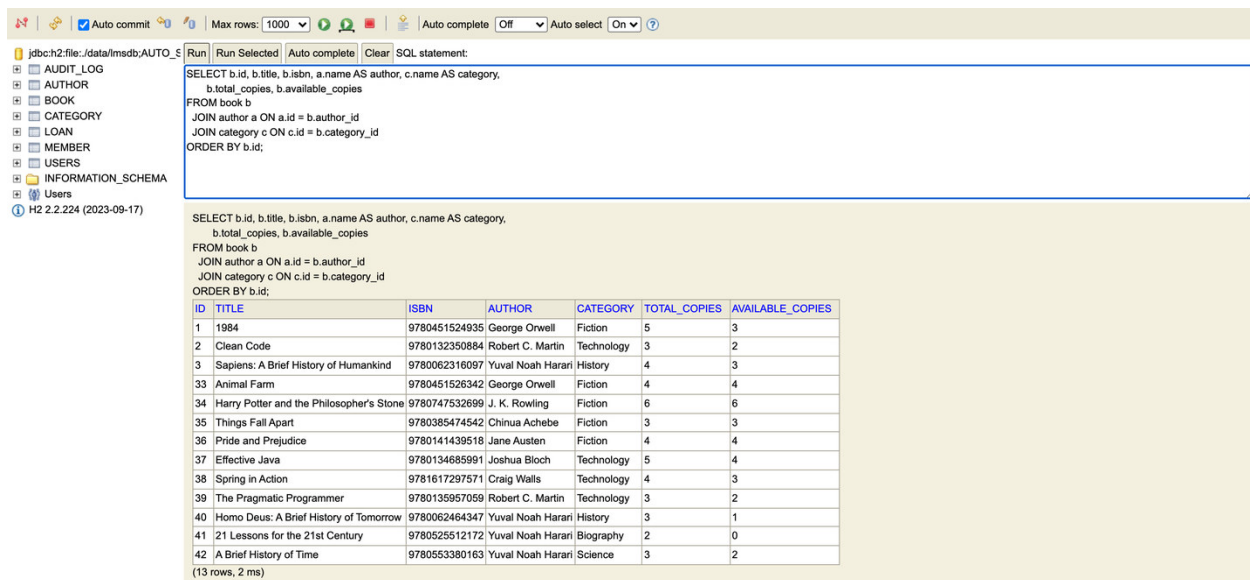


Figure 5.50: H2 console — book catalogue joined with author and category

ID	USERNAME	ACTION	ENTITY_NAME	DETAILS	TIMESTAMP
149	member	LOGIN	User	User logged in	2026-09-06 10:26:29.907691
148	librarian	LOGIN	User	User logged in	2026-09-06 10:26:26.768076
147	admin	LOGIN	User	User logged in	2026-09-06 10:25:55.864413
146	admin	CREATE	Loan	Issued book 21 Lessons for the 21st Century to Prakash Rai	2026-09-06 10:24:28.694018
145	admin	CREATE	Loan	Issued book Homo Deus: A Brief History of Tomorrow to Prakash Rai	2026-09-06 10:24:28.685369
144	admin	DELETE	Book	Deleted book id: 44	2026-09-06 10:24:28.665318
143	admin	UPDATE	Book	Updated book id: 44	2026-09-06 10:24:28.651732
142	admin	CREATE	Book	Created book: Domain-Driven Design	2026-09-06 10:24:28.621457
141	admin	LOGIN	User	User logged in	2026-09-06 10:24:28.468388
140	member	LOGIN	User	User logged in	2026-09-06 10:24:28.393003
139	admin	LOGIN	User	User logged in	2026-09-06 10:24:28.297702
138	admin	CREATE	Loan	Issued book 21 Lessons for the 21st Century to Prakash Rai	2026-09-06 10:23:40.596312

Figure 5.51: H2 console — persisted audit_log rows

6. TESTING AND DEBUGGING

Testing was an essential phase of the project and was carried out to confirm that every functional requirement behaves as specified. Three complementary approaches were adopted.

- **Unit testing** — Focused on individual components in isolation — validation annotations on DTOs and entities, repository query methods, the fine calculation in the loan service and the token generation and validation logic in JwtUtil.
- **Integration testing** — Verified the flow of data between controller, service, repository and database, ensuring that CRUD operations, transaction boundaries and lazy-loaded associations behave correctly when the layers are exercised together.
- **System testing** — Evaluated complete end-to-end workflows through the running application: authenticating, obtaining a token, managing the catalogue, issuing and returning a book, generating reports and confirming that access control blocks unauthorised operations.

Debugging was carried out using application log output, the H2 console for direct inspection of the persisted rows, Swagger UI for exercising individual endpoints, and browser developer tools for the web client. Several defects were identified and corrected during this phase. The most significant concerned lazy-loaded associations: because the application disables the open-session-in-view pattern (spring.jpa.open-in-view=false), components that read a lazily fetched association outside an active persistence context failed with a LazyInitializationException. This affected the overdue scheduler, which reads the member and book of each overdue loan, and both report generators, which read the author, category and member of each row. The defect was corrected by declaring the affected methods `@Transactional` so that the persistence context stays open for the duration of the operation.

A second defect was a request mapping mismatch, in which the audit log controller was mapped to /api/audit while the security configuration and the web client both referenced /api/audit-logs; the controller mapping was corrected. Both classes of defect were found precisely because the test cases below were executed against real data rather than assumed to pass.

6.1. Test Cases

The tables below record the test cases executed against the running application. Every case was run manually through Swagger UI, the web client or a direct HTTP request, and the observed result was compared with the expected outcome.

Table 6.1: Authentication and Authorisation Test Cases

S.N.	Scenario	Action	Expected Outcome	Status
1	Admin login	Send POST /api/auth/login with username 'admin' and password 'Admin@123'	Authentication succeeds and a signed JWT is returned with role ADMIN	PASS
2	Librarian login	Send POST /api/auth/login with username 'librarian' and password 'Lib@1234'	Authentication succeeds and a JWT is returned with role LIBRARIAN	PASS
3	Member login	Send POST /api/auth/login with username 'member' and password 'Member@123'	Authentication succeeds and a JWT is returned with role MEMBER	PASS
4	Invalid credentials	Send POST /api/auth/login with an incorrect password	System returns 401 Unauthorized with the message 'Invalid username or password'	PASS
5	Access without a token	Send GET /api/members with no Authorization header	System rejects the request with 403 Forbidden	PASS
6	Insufficient role	A MEMBER sends DELETE /api/books/33 with a valid member token	System rejects the request with 403 Forbidden	PASS
7	Password storage	Inspect the USERS table through the H2 console	Passwords are stored only as BCrypt hashes, never in plain text	PASS

Table 6.2: Book CRUD Test Cases

S.N.	Scenario	Action	Expected Outcome	Status
1	View all books	Send GET /api/books	A paginated list of books is returned with author and category names resolved	PASS
2	View a single book	Send GET /api/books/37 using a valid book ID	The requested book details are returned	PASS
3	View a non-existent book	Send GET /api/books/9999 using an invalid ID	System returns 404 Not Found with an appropriate error message	PASS
4	Create a book	ADMIN sends POST /api/books with valid book information	The book is created and returned with a generated ID	PASS
5	Create with invalid data	ADMIN submits an empty title and ISBN, a publication year of 1200 and a negative copy count	Validation fails and 400 Bad Request is returned with a field-by-field breakdown	PASS
6	Update a book	ADMIN sends PUT /api/books/{id} with valid updated information	The existing book record is updated successfully	PASS
7	Delete a book	ADMIN sends DELETE /api/books/{id} using a valid ID	The book is deleted and a success response is returned	PASS
8	Delete a non-existent book	ADMIN sends DELETE /api/books/{id} using an invalid ID	System returns 404 Not Found	PASS
9	Duplicate ISBN	ADMIN creates a book using an ISBN that already exists	The unique constraint is enforced and the operation is rejected	PASS

Table 6.3: Search, Filter, Sort and Pagination Test Cases

S.N.	Scenario	Action	Expected Outcome	Status
1	Search by title	Send GET /api/books?title=java	Only books whose title contains 'java' are returned, case-insensitively	PASS
2	Filter by category	Send GET /api/books?categoryId=3	Only books belonging to the Technology category are returned	PASS
3	Filter by availability	Send GET /api/books?availableOnly=true	Only books with at least one available copy are returned	PASS
4	Combined filters	Send GET /api/books?categoryId=3 &availableOnly=true	Both predicates are applied together by the Specification	PASS
5	Sort ascending	Send GET /api/books?sort=title,asc	Books are returned in ascending order of title	PASS
6	Sort descending	Send GET /api/books?sort=publicationYear,desc	Books are returned newest first	PASS
7	Pagination	Send GET /api/books?page=0&size=3	At most three records are returned together with totalElements, totalPages and page metadata	PASS

Table 6.4: Loan and Circulation Test Cases

S.N.	Scenario	Action	Expected Outcome	Status
1	Issue a book	LIBRARIAN sends POST /api/loans/issue with a valid book and member ID	A loan is created with a due date 14 days ahead and the available copy count is decremented	PASS
2	Issue when no copies remain	Issue a book whose availableCopies is zero	System returns 400 Bad Request with the message 'No available copies'	PASS
3	Issue to a suspended member	Issue a book to a member whose status is SUSPENDED	The request is rejected with 400 Bad Request	PASS
4	Return a book on time	Send PUT /api/loans/{id}/return before the due date	Status becomes RETURNED, the copy count is restored and no fine is charged	PASS
5	Return a book late	Send PUT /api/loans/{id}/return six days after the due date	Status becomes RETURNED and a fine of NPR 60 (6 days × NPR 10) is calculated	PASS
6	Return an already-returned loan	Send PUT /api/loans/{id}/return for a RETURNED loan	System returns 400 Bad Request stating the loan has already been returned	PASS
7	Overdue detection	Run the scheduled job with loans whose due date has passed	Affected loans are marked OVERDUE and a reminder is dispatched for each	PASS

Table 6.5: Additional Feature Test Cases

S.N.	Scenario	Action	Expected Outcome	Status
1	Upload a book cover	ADMIN sends POST /api/books/{id}/cover as multipart form-data with a valid image file	The file is stored and its generated path is associated with the book	PASS
2	Download a book cover	Request the stored cover path under /uploads/	The image is served successfully as a static resource	PASS
3	Oversized upload	Upload a file larger than the configured 5 MB limit	System returns 400 Bad Request stating the maximum allowed size	PASS
4	PDF report	ADMIN sends GET /api/reports/loans/pdf	A formatted PDF circulation report is returned as an attachment	PASS
5	CSV report	ADMIN sends GET /api/reports/books/csv	A CSV inventory report containing every book is returned as an attachment	PASS
6	Report access control	A MEMBER requests GET /api/reports/books/csv	The request is rejected with 403 Forbidden	PASS
7	Audit trail	ADMIN sends GET /api/audit-logs after performing create, update and delete operations	Every write operation is listed with the acting username, action, entity and timestamp	PASS
8	Audit access control	A LIBRARIAN requests GET /api/audit-logs	The request is rejected with 403 Forbidden as the endpoint is ADMIN-only	PASS
9	API documentation	Open /swagger-ui.html in a browser	Interactive documentation for every endpoint is rendered and can be exercised	PASS

7. DISCUSSION

The Library Management System achieved its stated objectives. Administrators and librarians can authenticate, manage the catalogue of books, authors and categories, register and maintain members, and issue and return books, while members can browse and search the catalogue. The system also provides book cover upload and download, exportable PDF and CSV reports, a dashboard summarising library statistics and a complete audit trail. Data security is maintained through Spring Security, BCrypt password hashing, stateless JWT authentication and three-tier role-based authorisation.

The four-layer architecture proved valuable well beyond its academic justification. Because controllers deal only with HTTP concerns and services own the business rules and transaction boundaries, features such as fine calculation and audit logging could be added inside the service layer without touching a single controller. Using DTOs at the API boundary rather than exposing entities directly prevented lazy-loading artefacts from leaking into JSON responses and kept the public contract stable while the underlying entities evolved.

Several challenges were encountered during development. Configuring stateless JWT authentication required care: the filter chain had to be set to `SessionCreationPolicy.STATELESS`, CSRF protection disabled in favour of token authentication, and the custom `JwtAuthFilter` registered before the standard `UsernamePasswordAuthenticationFilter` so that the security context is populated before authorisation is evaluated. Getting the ordering wrong produced authentication failures that were not obvious from the response alone.

The most instructive difficulty concerned lazy loading. The application deliberately sets `spring.jpa.open-in-view` to `false`, which is the recommended production setting because it prevents database connections from being held open for the whole duration of a web request. The consequence is that any code touching a lazily fetched association outside an active persistence context fails at runtime. This surfaced in three places — the overdue scheduler and both report generators — and was resolved by making those operations transactional. It was a useful demonstration that a configuration chosen for good architectural reasons imposes real obligations on the code written against it, and that such defects only appear once the feature is exercised with representative data.

Implementing search, filtering, sorting and pagination through Spring Data Specifications rather than a proliferation of query methods was also worthwhile. A single endpoint accepts optional title, author, category and availability parameters in any combination, and the Specification composes only the predicates actually supplied. Adding a further filter requires one small Specification method rather than a combinatorial expansion of repository queries.

The final output is a functional REST-based Library Management System capable of securely managing a library's catalogue, members and circulation. The screenshots presented in Section 5

document the authentication flow, role-based access control, CRUD operations, search and filtering, file upload, loan issue and return with automatic fine calculation, the scheduled overdue job, report generation and the audit trail, and together demonstrate a fully operational system.

8. FUTURE ENHANCEMENTS AND CONCLUSION

8.1. Future Enhancements

Although the current system fulfils every stated requirement, it can be extended considerably.

- **Reservation and hold queue** — Allow members to reserve a title that is currently on loan and be notified automatically when a copy is returned.
- **Fine payment and receipting** — Record fine payments against a loan, issue receipts and report on outstanding balances.
- **Refresh-token rotation** — Issue short-lived access tokens alongside rotating refresh tokens to reduce the impact of a leaked token.
- **Production database** — Migrate from file-based H2 to MySQL or PostgreSQL with Flyway-managed schema migrations.
- **Automated test suite** — Add JUnit 5 and Spring Boot test coverage — including `@DataJpaTest` slices and `MockMvc` integration tests — so that the manual test cases in Section 6 are executed on every build.
- **Caching** — Introduce Redis or the Spring cache abstraction for frequently repeated catalogue queries.
- **Richer analytics** — Extend the dashboard with circulation trends, most-borrowed titles and member activity over time.
- **Containerised deployment** — Package the application and its database as Docker images for reproducible deployment.
- **Barcode and RFID support** — Extend the client to accept handheld scanner input for rapid check-in and check-out.

8.2. Conclusion

The Library Management System successfully demonstrates the practical application of Advanced Java Programming concepts using the Spring Boot framework. The system was designed as a RESTful backend application with a clear layered architecture consisting of Controller, Service, Repository and Entity components. This separation of responsibilities makes the application easier to understand, maintain, test and extend, and it allowed several features to be added during development without disturbing existing code.

All mandatory requirements were met. The system provides complete CRUD operations across five domain entities, persists them through Spring Data JPA and an H2 relational database, validates every input server-side, handles errors centrally, and secures the API using Spring Security with stateless JWT authentication and three-tier role-based authorisation. Search, filtering, sorting and pagination

make retrieval efficient as the catalogue grows.

Five additional features were implemented, exceeding the required minimum of three: book cover upload and download, a scheduled overdue-detection job with e-mail notification, PDF report generation using iText, CSV report generation using OpenCSV, and a persistent audit trail reinforced by a Spring AOP aspect. Interactive documentation is published automatically through SpringDoc OpenAPI.

Testing the system against realistic data proved as valuable as building it. Three genuine defects — two lazy-initialisation failures and a request-mapping mismatch — were discovered only because the additional features were exercised end-to-end rather than assumed to work, and all three were corrected before submission. This reinforced the importance of verifying behaviour against real data instead of relying on code that merely compiles.

Overall the project meets its objectives while remaining small enough to be understood in full. It provides a solid foundation for the future enhancements described above, including reservations, fine payment, a production database, an automated test suite and containerised deployment.

9. REFERENCES

- [1] D. Yadav, A. Singh and M. Dubey, “Library Management System using Java and MySQL,” International Journal of Computer Techniques, vol. 13, no. 2, IJCT-V13I2P93, March–April 2026. Available: <https://ijctjournal.org/library-management-system-java-mysql/>
- [2] X. Guo, T. Hong, Z. Chen and M. Kadoch, “Design and Implementation of Web-Based Library Management System,” IEEE, 2023. Available: <https://pure.etsmtl.ca/en/publications/design-and-implementation-of-web-based-library-management-system/>
- [3] M. Siva and S. Gnanapriya, “Library Book Management System using Spring Boot,” October 2025. Available: https://www.researchgate.net/publication/397120256_Library_book_Management_System_using_Spring_Boot
- [4] IJCSE, “Object-Oriented Design Principles and GoF Patterns in a Spring Boot Library Management System: Architecture, Implementation and Evaluation,” International Journal of Computer Science and Engineering, vol. 10, no. 2, IJCSE-V10I2P21, April 2026. Available: <https://www.ijcsejournal.org/object-oriented-gof-spring-boot/>
- [5] Auth0, “JSON Web Tokens Introduction.” Available: <https://jwt.io/introduction>
- [6] GeeksforGeeks, “Spring Boot Tutorial,” May 2026. Available: <https://www.geeksforgeeks.org/advance-java/spring-boot/>
- [7] Spring, “Spring Data JPA.” Available: <https://spring.io/projects/spring-data-jpa>
- [8] Spring, “Spring Security Reference.” Available: <https://docs.spring.io/spring-security/reference/>
- [9] SpringDoc, “OpenAPI 3 Library for Spring Boot.” Available: <https://springdoc.org/>
- [10] H2 Database Engine, “H2 Documentation.” Available: <https://www.h2database.com/html/main.html>